REPUBLIQUE DU CAMEROUN
****
PAIX-TRAVAL-PATRIE
****
UNIVERSITE DE DSCHAN
****
FACULTES DES SCIENCES
****
DEPARTEMENT DE SCIENCES

REPUBLIC OF CAMEROON
****
PEACE-WORK-FATHERLAND
****
UNIVERSITY OF DSCHANG
****
FACULTY OF SCIENCES
****
DEPARTEMENT OF SCIENCES

Architecture des Systèmes Intelligents

Combinaison de classeurs, explicabilité des modèles

Présenté par :

DNJOMOU YONMBA WILFRIED LOIC	**CM-UDS-24SCI0999**
KENFACK LEONEL	**CM-UDS-17SCI0998**
SAGUEU WAKAM DILANE	**CM-UDS-24SCI1040**
NZOGNING MBONDA SOCRATE	**CM-UDS-19SCI2910**

Superviseur : Pr. KENGNE TCHENDJI Vianney

Année académique 2025-2026

Table des matières

1 Introduction générale 5
2 Combinaison de classifieurs 6
2.1 Introduction 6
2.2 Définition et phases de la combinaison de classifieurs 6
Nature des sorties et niveaux d'information 7
2.3 Motivations théoriques et justifications statistiques 7
Théorème du jury de Condorcet 7
Décomposition biais–variance 8
Théorie unifiée de la diversité dans les ensembles 8
Les problèmes de Dietterich 9
2.4 Achirtectures des systèmes multi-classifieurs 10
Architectures de contrôle de flux 10
Systèmes de sélection dynamique (DS) 11
2.5 Typologies des méthodes d'ensemble 11
Bagging (bootstrap aggregating) 11
Boosting 15
Stacking (stacked generalization) 18
Voting (vote majoritaire vs vote probabiliste) 24
2.6 Analyse comparative des approches 25
2.7 Avantages et limites de la combinaison 25
Avantages 25
Limites 25
2.8 Conclusion 26
3 Explicabilité des modèles 27
3.1 Introduction 27
3.2 Définition de l'explicabilité 27
Formulation conceptuelle 28
Positionnement systémique 28
3.3 Interprétabilité vs explicabilité 29
Différence structurelle fondamentale 29
Typologie formelle des modèles 30
Conséquence scientifique majeure 30
3.4 Méthodes d'explicabilité 31
Modèles intrinsèquement interprétables 31

Méthodes post-hoc . . . 31
3.5 Explicabilité locale vs globale . . . 34
Explicabilité locale . . . 34
Explicabilité globale . . . 34
Différences structurelles . . . 35
3.6 Conclusion — Importance décisionnelle, enjeux éthiques et réglementaires 35
4 Combinaison de Classifieurs et Explicabilité . . . 37
4.1 Le Problème de la Boîte Noire dans les Ensembles . . . 37
Qu'est-ce qu'une boîte noire en apprentissage automatique ? . . . 37
Pourquoi les ensembles de classifieurs deviennent-ils des boîtes noires ? . . . 37
L'effet cascade de l'agrégation . . . 38
4.2 Complexité des Ensembles de Modèles . . . 38
La dimension de Vapnik-Chervonenkis et l'explosion de la capacité . . . 38
La décomposition biais-variance et le compromis d'opacité . . . 39
Le théorème de Condorcet et ses implications pour l'explicabilité . . . 39
L'identité de Krogh-Vedelsby : diversité et erreur d'ensemble . . . 40
L'accumulation des résidus dans le Boosting . . . 40
4.3 Stratégies d'Explicabilité des Ensembles . . . 41
Méthodes post-hoc : limites de LIME sur les ensembles arborescents . . . 41
TreeSHAP : la solution exacte fondée sur la théorie des jeux . . . 42
SIRUS : extraction de règles stables depuis les forêts aléatoires . . . 43
Les architectures Glass-Box : transparence par conception . . . 45
Sélection Dynamique de Classifieurs (DCS/DES) et explicabilité contextuelle 47
XStacking : le méta-apprentissage guidé par l'explication . . . 47
4.4 Compromis Performance–Compréhension . . . 48
Le mythe du compromis strict et sa déconstruction empirique . . . 48
L'effet Rashomon et la multiplicité des modèles équivalents . . . 49
Le Consensus XAI : agréger les explications pour réduire leur variance . . . 50
Guide de décision : quelle stratégie adopter ? . . . 51
4.5 Conclusion . . . 51
5 Conclusion générale . . . 53

Table des figures

2.1 Architecture parallèle pour un système multi-classifieurs. 10
2.2 Architecture séquentielle pour un système multi-classifieurs. 11
2.3 Schéma pédagogique du bagging : du jeu de données original aux échantillons bootstrap, puis aux modèles de base et à l'agrégation. 13
2.4 Aperçu de l'algorithme Random Forest. 14
2.5 Schéma pédagogique du boosting : chaîne de modèles où chaque étape corrige les erreurs de la précédente. 15
2.6 Schéma pédagogique du stacking : architecture en couches avec apprenants de niveau 0 et méta-modèle de niveau 1. 19

Liste des tableaux

2.1 Comparaison des niveaux d'information en sortie des classifieurs 7
2.2 Comparaison synthétique bagging / boosting / stacking 25
3.1 Différences clés entre interprétabilité et explicabilité 29
3.2 Comparaison explicabilité locale vs globale 35
4.1 Comparaison des méthodes SHAP pour les ensembles arborescents 43
4.2 Comparaison entre Gradient Boosting (XGBoost) et EBM 46
4.3 Comparaison Stacking conventionnel vs XStacking 48
4.4 Comparaison AUC — EBM vs XGBoost sur des jeux de données réels 49
4.5 Guide de sélection de la stratégie d'explicabilité selon le contexte 51

1 Introduction générale

L'intelligence artificielle et l'apprentissage automatique occupent une place croissante dans les systèmes de décision, industriels, médicaux, financiers ou réglementaires. La **combinaison de classifieurs**, ou apprentissage d'ensemble (*ensemble learning*), s'impose comme un paradigme central : on construit un comité de modèles dont les prédictions sont agrégées pour améliorer la précision et la robustesse (bagging, boosting, stacking). Parallèlement, l'**explicabilité des modèles** est devenue une exigence majeure : les décisions automatisées sont scrutées par les utilisateurs, les régulateurs et la société ; comprendre *pourquoi* un modèle prédit relève de la confiance, de la conformité et de l'éthique. Les méthodes d'explicabilité (intrinsèques ou post-hoc : LIME, SHAP, importance des variables) visent à rendre les systèmes moins opaques. Ce rapport se situe à l'intersection de ces deux thématiques : les systèmes d'ensemble complexifient le modèle global et renforcent l'effet « boîte noire », tout en offrant des gains de performance substantiels.

Les méthodes d'ensemble améliorent la précision au prix d'une complexité accrue (plusieurs modèles, fusion, dépendances), ce qui soulève une question centrale : *comment concilier la puissance des systèmes combinant plusieurs classifieurs avec l'exigence d'explicabilité et d'interprétabilité des décisions ?* D'un côté, un classifieur unique peut être plus facile à expliquer ; de l'autre, un ensemble peut être plus performant mais plus difficile à interpréter. La problématique recouvre les dimensions techniques, architecturales et réglementaires (transparence, traçabilité).

Les objectifs sont de présenter les fondements de la combinaison de classifieurs (définitions, typologies, motivations théoriques, aspects architecturaux) ; de clarifier les concepts d'explicabilité et d'interprétabilité et les principales méthodes (intrinsèques et post-hoc : LIME, SHAP, PDP/ICE, explicabilité locale et globale) ; d'analyser les enjeux spécifiques aux ensembles (complexité, stratégies d'explication, compromis performance et compréhension) ; et de mettre en perspective les avantages et limites au regard de l'explicabilité.

Le document comporte trois parties : la **combinaison de classifieurs** (définition, typologies, aspects architecturaux, analyse comparative, avantages et limites) ; l'**explicabilité des modèles** (définition, interprétabilité, méthodes, enjeux dans les systèmes décisionnels, éthique et réglementation) ; la **combinaison et l'explicabilité** (boîte noire, stratégies, compromis performance et compréhension). La conclusion générale synthétise les apports et ouvre sur les perspectives.

2 Combinaison de classifieurs

2.1 Introduction

L'évolution de l'apprentissage automatique a franchi une étape décisive avec l'émergence des systèmes de classifieurs multiples (*Multiple Classifier Systems – MCS*), plus communément désignés sous le terme d'apprentissage d'ensemble [11, 33, 18]. Cette approche repose sur un paradigme fondamental : la combinaison stratégique de plusieurs modèles de prédiction pour pallier les insuffisances inhérentes à tout classifieur unique.

Historiquement, la recherche en intelligence artificielle s'est concentrée sur l'optimisation d'algorithmes individuels. Cependant, l'expérience a démontré qu'un modèle unique, aussi sophistiqué soit-il, est souvent limité par ses propres biais structurels et sa sensibilité au bruit des données. L'apprentissage d'ensemble propose de substituer au modèle solitaire un comité de décideurs dont la sagesse collective surpasse les capacités individuelles.

Cette section propose une exploration de la combinaison de classifieurs, structurée autour des fondements théoriques, des typologies méthodologiques et des considérations architecturales qui définissent ce domaine. L'analyse s'attache à montrer comment la diversité des modèles et la rigueur de leur agrégation permettent d'atteindre des niveaux de performance et de robustesse inaccessibles aux approches classiques.

2.2 Définition et phases de la combinaison de classifieurs

La **combinaison de classifieurs** est définie comme le processus d'intégration des sorties de plusieurs modèles d'apprentissage afin de générer une décision globale plus fiable et précise. Un système multi-classifieurs est structurellement composé d'un ensemble d'apprenants de base, souvent qualifiés d'*apprenants faibles* (*weak learners*) lorsqu'ils sont pris isolément, et d'un mécanisme de fusion qui orchestre leurs réponses.

Le cycle de vie d'un tel système se décompose généralement en trois phases critiques :

1. **Génération de l'ensemble** : cette phase initiale consiste à produire un pool de modèles de base. L'objectif est de créer des modèles qui ne sont pas seulement performants, mais surtout complémentaires. La complémentarité est assurée par l'introduction de diversité, que ce soit par l'utilisation de différents sous-ensembles de données, de paramètres variés pour un même algorithme, ou de familles d'algorithmes distinctes.
2. **Sélection ou élagage (*pruning*)** : une fois le pool généré, il peut être nécessaire de réduire la taille de l'ensemble. L'élagage vise à éliminer les modèles redondants ou ceux dont la performance dégrade la décision collective. Cette étape permet non seulement d'améliorer la précision globale mais aussi de réduire les coûts computationnels lors de l'inférence.
3. **Combinaison ou fusion** : la phase finale consiste à fusionner les prédictions des modèles sélectionnés pour obtenir la réponse ultime. Le choix de la fonction de combinaison dépend crucialement du type d'information fournie par les classifieurs de base (niveau d'information en sortie).

Nature des sorties et niveaux d'information

L'efficacité de la combinaison dépend de la richesse des informations transmises par les modèles individuels. On distingue classiquement trois niveaux de sortie :

— **Niveau « Classe » (hard output)** : le classifieur fournit uniquement l'étiquette de la classe prédite. C'est l'information la plus élémentaire, où chaque modèle agit comme un votant binaire.

— **Niveau « Rang »** : le classifieur produit une liste ordonnée des classes possibles, de la plus probable à la moins probable. Ce format est particulièrement utile lorsque la classe réelle ne figure pas au premier rang mais reste parmi les premières propositions.

— **Niveau « Mesure » (soft output)** : c'est le niveau le plus riche. Le classifieur associe à chaque classe une valeur numérique, telle qu'une probabilité ou un score de confiance. Cette mesure quantifie l'incertitude du modèle et permet des fusions pondérées beaucoup plus fines.

Le tableau suivant présente une comparaison structurelle de ces types d'informations en sortie.

Table 2.1 – Comparaison des niveaux d'information en sortie des classifieurs

Type de sortie	Information contenue	Complexité de fusion	Cas d'utilisation typique
Classe	Étiquette discrète uniquement	Faible	Vote majoritaire simple
Rang	Préférences relatives ordonnées	Moyenne	Méthode de Borda, systèmes de recommandation
Mesure	Probabilités, scores de confiance	Élevée	Moyenne pondérée, stacking, modèles bayésiens

2.3 Motivations théoriques et justifications statistiques

La supériorité des systèmes multi-classifieurs n'est pas une simple observation empirique ; elle s'appuie sur des piliers théoriques robustes, notamment le théorème du jury de Condorcet et la décomposition biais–variance [30, 1, 14].

Théorème du jury de Condorcet

Formulé à l'origine dans le contexte des sciences sociales, ce théorème constitue une base théorique fondamentale pour l'apprentissage d'ensemble [34, 7]. Il postule que si un groupe d'individus doit prendre une décision binaire (par exemple, classer un objet comme A ou B) et que chaque individu a une probabilité p supérieure à 0,5 de choisir la bonne réponse, alors la probabilité que la décision de la majorité soit correcte augmente avec le nombre d'individus.

Mathématiquement, si L est le nombre de classifieurs indépendants et p leur précision individuelle, la probabilité P que l'ensemble prenne la bonne décision par vote majoritaire tend vers 1 lorsque L tend vers l'infini. À l'inverse, si $p < 0{,}5$, l'ajout de classifieurs réduit la performance globale. Cette démonstration justifie l'utilisation d'« apprenants faibles » pour construire des systèmes puissants, à condition qu'ils soient plus performants qu'un tirage aléatoire et qu'ils soient suffisamment indépendants.

Décomposition biais–variance

D'un point de vue statistique, l'erreur de généralisation d'un modèle d'apprentissage peut être décomposée en trois composantes : le biais au carré, la variance et l'erreur irréductible (bruit) [32, 16]. La formule de l'erreur quadratique moyenne (MSE) est :

$$\mathbb{E}\big[(y - \hat{f}(x))^2\big] = \mathrm{Bias}\big[\hat{f}(x)\big]^2 + \mathrm{Var}\big[\hat{f}(x)\big] + \sigma^2, \tag{1}$$

où σ^2 représente le bruit inhérent aux données.

— Le **biais** mesure l'écart entre la prédiction moyenne du modèle et la valeur réelle. Un biais élevé est le signe d'un modèle trop simpliste qui ne capture pas la complexité des données (sous-apprentissage, *underfitting*).

— La **variance** mesure la dispersion des prédictions pour différents jeux de données d'apprentissage. Une variance élevée indique que le modèle est trop sensible aux fluctuations et au bruit des données (sur-apprentissage, *overfitting*).

L'apprentissage d'ensemble permet de naviguer dans ce compromis biais–variance. Les méthodes de type bagging agissent principalement en réduisant la variance par la moyenne de modèles complexes, tandis que les méthodes de boosting se concentrent sur la réduction du biais en agrégeant itérativement des modèles simples pour corriger leurs erreurs systématiques.

Théorie unifiée de la diversité dans les ensembles

Une contribution théorique importante aux méthodes d'ensemble consiste à faire apparaître explicitement la diversité comme composante de la décomposition de la perte. Pour une vaste famille de fonctions de perte (notamment les divergences de Bregman, qui incluent l'erreur quadratique moyenne et l'entropie croisée), on peut montrer que la perte de l'ensemble s'exprime comme la moyenne des pertes individuelles diminuée d'un terme de diversité [24].

Dans le cas particulier de l'erreur quadratique moyenne, cette idée est capturée par l'identité de Krogh et Vedelsby [23]. Pour un ensemble de M modèles dont les prédictions sont $q_1(x), \ldots, q_M(x)$ pour une entrée x et une cible y, on pose

$$\bar{q}(x) = \frac{1}{M}\sum_{i=1}^{M} q_i(x).$$

L'identité s'écrit alors :

$$\big(\bar{q}(x) - y\big)^2 = \frac{1}{M}\sum_{i=1}^{M}\big(q_i(x) - y\big)^2 - \frac{1}{M}\sum_{i=1}^{M}\big(q_i(x) - \bar{q}(x)\big)^2. \tag{2}$$

Le membre de gauche est l'erreur quadratique de l'ensemble, le premier terme à droite la moyenne des erreurs quadratiques individuelles, et le second terme (souvent appelé « ambiguïté » ou « diversité ») mesure le degré de désaccord entre les prédictions des modèles. L'identité montre que, toutes choses égales par ailleurs, l'erreur de l'ensemble diminue lorsque la diversité augmente.

Démonstration de l'identité de Krogh et Vedelsby. Pour simplifier les notations, on note $\bar{q} = \bar{q}(x)$ et $q_i = q_i(x)$.

(i) Développement de l'erreur de l'ensemble.

$$(\bar{q} - y)^2 = \bar{q}^2 - 2\bar{q}y + y^2.$$

(ii) Développement de la moyenne des erreurs individuelles.

$$\begin{aligned}
\frac{1}{M}\sum_{i=1}^{M}(q_i - y)^2 &= \frac{1}{M}\sum_{i=1}^{M}(q_i^2 - 2q_i y + y^2) && (3)\\
&= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2y\left(\frac{1}{M}\sum_{i=1}^{M} q_i\right) + y^2 && (4)\\
&= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2y\bar{q} + y^2. && (5)
\end{aligned}$$

(iii) Développement du terme de diversité.

$$\begin{aligned}
\frac{1}{M}\sum_{i=1}^{M}(q_i - \bar{q})^2 &= \frac{1}{M}\sum_{i=1}^{M}(q_i^2 - 2q_i\bar{q} + \bar{q}^2) && (6)\\
&= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2\bar{q}\left(\frac{1}{M}\sum_{i=1}^{M} q_i\right) + \bar{q}^2 && (7)\\
&= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2\bar{q}^2 + \bar{q}^2 && (8)\\
&= \frac{1}{M}\sum_{i=1}^{M} q_i^2 - \bar{q}^2. && (9)
\end{aligned}$$

(iv) Soustraction des deux termes.

$$\begin{aligned}
\frac{1}{M}\sum_{i=1}^{M}(q_i - y)^2 - \frac{1}{M}\sum_{i=1}^{M}(q_i - \bar{q})^2 &= \left(\frac{1}{M}\sum_{i=1}^{M} q_i^2 - 2y\bar{q} + y^2\right) - \left(\frac{1}{M}\sum_{i=1}^{M} q_i^2 - \bar{q}^2\right) && (10)\\
&= -2y\bar{q} + y^2 + \bar{q}^2 && (11)\\
&= (\bar{q} - y)^2. && (12)
\end{aligned}$$

On retrouve bien l'identité annoncée. Comme le terme de diversité est une somme de carrés, il est toujours positif ou nul. L'erreur de l'ensemble est donc toujours inférieure ou égale à la moyenne des erreurs individuelles, avec égalité lorsque tous les modèles prédisent la même valeur (diversité nulle).

Cette identité justifie l'importance de promouvoir la diversité dans les ensembles (via le bagging, le boosting, la randomisation des architectures, etc.) et ouvre sur des généralisations à d'autres pertes au moyen des divergences de Bregman [24].

Les problèmes de Dietterich

Thomas Dietterich a identifié trois raisons fondamentales expliquant pourquoi un ensemble peut surpasser un modèle unique :

1. **Le problème statistique** : lorsque le volume de données est faible par rapport à la taille de l'espace des hypothèses, l'algorithme peut trouver plusieurs modèles qui s'ajustent parfaitement aux données d'entraînement. En les combinant, l'ensemble réduit le risque de choisir par erreur un modèle qui généralise mal.
2. **Le problème computationnel** : de nombreux algorithmes (comme les réseaux de neurones) utilisent des heuristiques de recherche locale qui peuvent rester bloquées dans des optima locaux. Un ensemble effectuant plusieurs recherches à partir de points de départ différents a plus de chances de trouver une meilleure approximation globale.
3. **Le problème de représentation** : parfois, la véritable fonction cible ne peut être représentée par aucun des modèles de l'espace d'hypothèses choisi. La combinaison de plusieurs modèles permet d'étendre cet espace et d'approcher des fonctions plus complexes.

2.4 Achirtectures des systèmes multi-classifieurs

L'architecture d'un système multi-classifieurs définit la manière dont les informations circulent entre les composants et la façon dont les ressources sont exploitées.

Architectures de contrôle de flux

On distingue trois organisations principales :

1. **Architecture parallèle** : tous les classifieurs opèrent indépendamment et simultanément. La fusion intervient à la fin. Cette structure est neutre (aucun classifieur n'influence les autres) et hautement parallélisable sur du matériel multi-cœurs.
2. **Architecture séquentielle (en série)** : les classifieurs sont organisés en cascade. Un premier classifieur rapide effectue une prédiction ; s'il échoue ou exprime une faible confiance, l'exemple est passé à un second modèle plus complexe. Cela permet d'économiser des ressources de calcul en ne sollicitant les modèles coûteux que pour les cas difficiles.
3. **Architecture hybride** : elle combine les deux approches. Par exemple, un premier étage parallèle peut générer un consensus, dont le résultat est validé par un classifieur de contrôle séquentiel.

Ces formes peuvent être illustrées par des figures dédiées aux architectures parallèle et séquentielle.

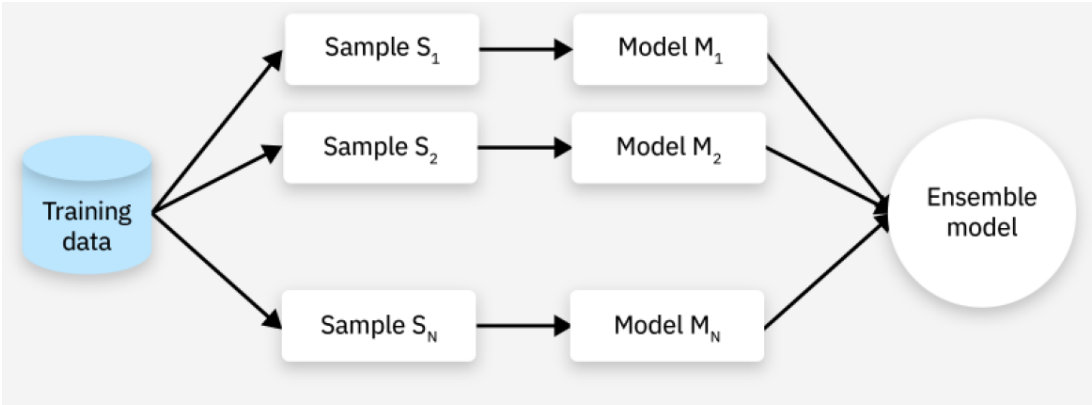


FIGURE 2.1 – Architecture parallèle pour un système multi-classifieurs.

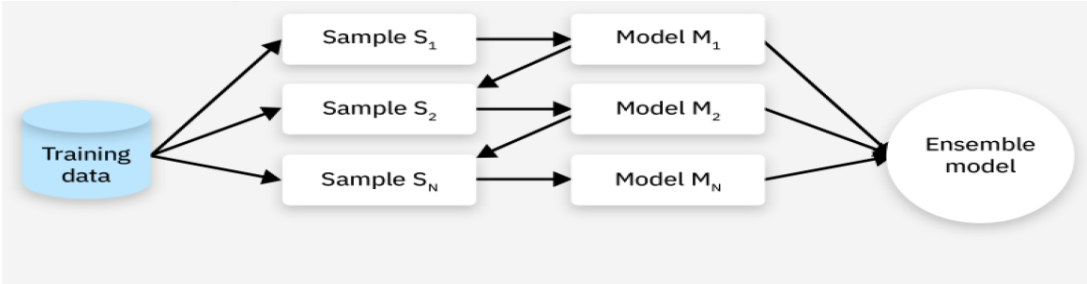


FIGURE 2.2 – Architecture séquentielle pour un système multi-classifieurs.

Systèmes de sélection dynamique (DS)

Une tendance majeure consiste à ne pas utiliser tous les classifieurs pour chaque exemple, mais à sélectionner « à la volée » le ou les modèles les plus compétents [9, 28, 22, 8, 10].

— **Logique de compétence locale** : on définit une « région de compétence » autour d'un exemple test (en utilisant par exemple ses plus proches voisins dans le jeu d'entraînement). On évalue ensuite quels modèles du pool ont été les plus performants dans cette zone spécifique.

— **DCS (dynamic classifier selection)** : on choisit l'unique meilleur modèle pour l'exemple donné.

— **DES (dynamic ensemble selection)** : on construit un sous-ensemble de modèles experts pour cette région particulière.

2.5 Typologies des méthodes d'ensemble

Les méthodes d'ensemble se divisent principalement en approches parallèles (bagging), séquentielles (boosting) et hiérarchiques (stacking) [4, 36, 26].

Bagging (bootstrap aggregating)

Le bagging (pour *bootstrap aggregating*) vise à réduire la variance en entraînant plusieurs modèles indépendants en parallèle sur des échantillons de données légèrement différents. Introduit par Breiman [2] et largement diffusé ensuite [19, 17, 1], il est particulièrement efficace pour des apprenants de base instables comme les arbres de décision profonds.

Définition. À partir d'un ensemble d'apprentissage $D = \{(x_j, y_j)\}_{j=1}^n$, le bagging construit m échantillons bootstrap D_i par tirage aléatoire avec remise, chacun de taille n. Sur chaque D_i, on entraîne un modèle de base $\hat{f}_i$ (*base learner*). À la prédiction, les sorties des m modèles sont agrégées :

— en régression, par moyenne

$$\hat{f}(x) = \frac{1}{m} \sum_{i=1}^{m} \hat{f}_i(x);$$

— en classification, par vote majoritaire

$$\hat{y}(x) = \arg\max_k \sum_{i=1}^{m} \mathbb{I}\big(\hat{y}_i(x) = k\big).$$

La construction des m modèles étant indépendante, l'entraînement est naturellement parallélisable et s'adapte bien aux architectures multi-cœurs ou distribuées [4, 19].

Fonctionnement détaillé. On peut résumer la procédure de bagging en quatre étapes [2, 25] :

1. **Choix de l'apprenant de base et du nombre de modèles** : on fixe un modèle de base (par exemple, un arbre de décision) et un nombre de réplications m (souvent entre 50 et 500).
2. **Génération des échantillons bootstrap** : pour chaque $i \in \{1, \ldots, m\}$, on tire n exemples de D avec remise pour former D_i. Chaque observation a une probabilité $1/n$ d'être choisie à chaque tirage ; au total, environ 63,2 % des exemples distincts apparaissent dans D_i, les autres constituant les données *out-of-bag*.
3. **Entraînement des modèles** : on entraîne indépendamment un modèle $\hat{f}_i$ sur chaque D_i. Cette étape est triviale à paralléliser.
4. **Agrégation des prédictions** : pour une nouvelle entrée x, on calcule les prédictions des m modèles et on les agrège par moyenne (régression) ou vote (classification), comme indiqué ci-dessus.

Un pseudocode simplifié est le suivant :

```
fonction Bagging(D, m, base_learner):
    modèles = []
    pour i de 1 à m:
        D_i = bootstrap_sample(D)      // tirage avec remise
        f_i = base_learner.fit(D_i)
        modèles.ajouter(f_i)
    retour modèles

fonction Predire(modèles, x):
    prédictions = [f_i.predict(x) pour f_i dans modèles]
    si régression:      retour moyenne(prédictions)
    si classification:  retour mode(prédictions)
```

Rôle des données out-of-bag (OOB). Lors de la génération de D_i, chaque exemple de D a une probabilité $(1 - 1/n)^n \simeq 1/e$ de ne jamais être sélectionné ; environ 36,8 % des observations restent donc en dehors de chaque bootstrap et sont dites *out-of-bag* pour le modèle $\hat{f}_i$ considéré.

Pour une observation (x_j, y_j), on note

$$S_j = \{\, i \in \{1, \ldots, m\} \mid (x_j, y_j) \notin D_i \,\}$$

l'ensemble des indices de modèles qui n'ont pas vu ce point à l'entraînement. On peut alors définir une prédiction OOB par

$$\tilde{f}_j(x_j) = \frac{1}{|S_j|} \sum_{i \in S_j} \hat{f}_i(x_j)$$

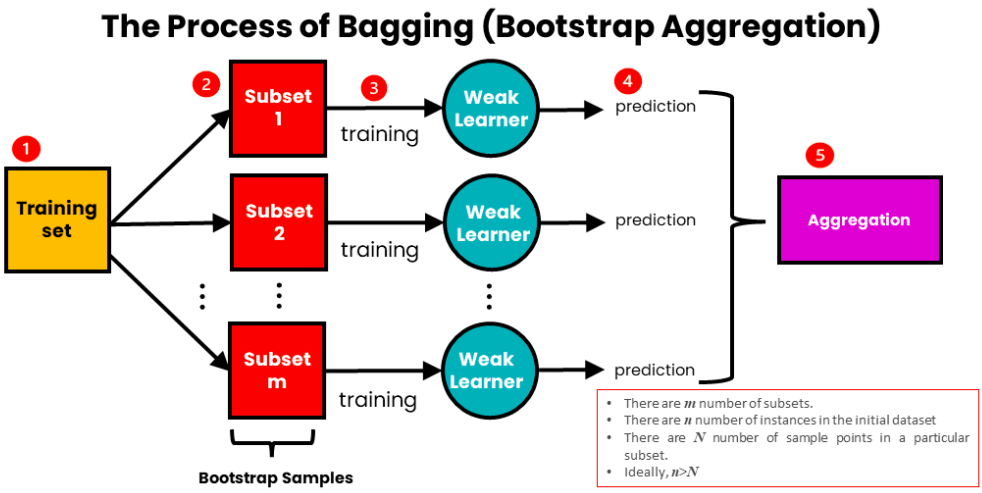


FIGURE 2.3 – Schéma pédagogique du bagging : du jeu de données original aux échantillons bootstrap, puis aux modèles de base et à l'agrégation.

en régression, ou par vote majoritaire restreint aux $i \in S_j$ en classification. Une estimation de l'erreur OOB est donnée par

$$\varepsilon_{\text{OOB}} = \frac{1}{n} \sum_{j=1}^{n} \ell\big(\tilde{f}_j(x_j), y_j\big),$$

où ℓ est la fonction de perte (MSE, entropie croisée, etc.). Cette quantité fournit une approximation fiable de l'erreur de généralisation, sans avoir besoin de réserver un jeu de validation distinct [2, 25, 19].

Réduction de la variance. Le principal intérêt du bagging est de réduire la variance des prédictions. Dans un scénario idéal où les modèles $\hat{f}_i$ seraient indépendants et de variance v, la variance de la moyenne

$$\hat{f}(x) = \frac{1}{m} \sum_{i=1}^{m} \hat{f}_i(x)$$

serait v/m. En pratique, les modèles ne sont pas indépendants ; si l'on note ρ la corrélation moyenne entre deux prédicteurs, on obtient approximativement [7] :

$$\operatorname{Var}\big[\hat{f}(x)\big] \approx \rho v + \frac{1-\rho}{m}\, v.$$

Quand m augmente, le second terme décroît comme $1/m$ et la variance tend vers ρv. Le bagging est donc particulièrement pertinent lorsque les modèles de base sont peu biaisés mais très variables (comme les arbres profonds) : il stabilise leurs prédictions sans augmenter significativement le biais.

Exemples d'algorithmes baggés. *Forêt aléatoire.* La forêt aléatoire (*Random Forest*) est l'extension la plus célèbre du bagging. En plus du bootstrapping, elle introduit une randomisation des caractéristiques (*features*) : à chaque division d'un nœud de l'arbre, seul un sous-ensemble

aléatoire de variables est considéré. Cela décorrèle les arbres et renforce encore la réduction de variance [2, 19].

Un aperçu schématique de l'algorithme Random Forest est donné par la figure 2.4.

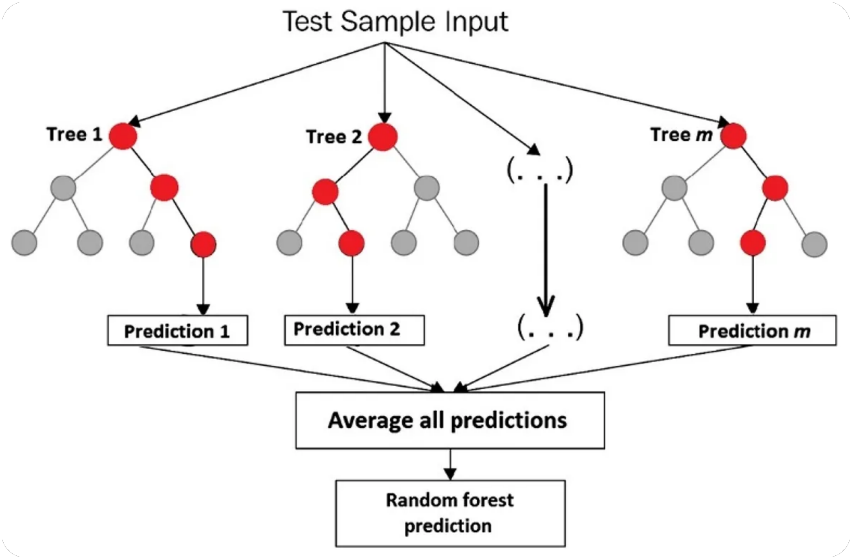


FIGURE 2.4 – Aperçu de l'algorithme Random Forest.

Exemple : bagging avec k plus proches voisins. Le bagging ne se limite pas aux arbres. Par exemple, appliqué au classifieur des k plus proches voisins (k-NN), il permet de stabiliser un modèle naturellement très sensible aux fluctuations locales des données. On génère m jeux bootstrap D_i, chacun servant de base à un modèle k-NN (qui consiste essentiellement à mémoriser D_i). Pour une nouvelle observation x, on agrège les m prédictions k-NN par vote ou moyenne des probabilités estimées [14, 4].

Dans les bibliothèques modernes comme scikit-learn, ces variantes sont disponibles via la classe `BaggingClassifier` en choisissant simplement l'estimateur de base (arbre, k-NN, etc.) et en activant le calcul de la « OOB score » pour obtenir automatiquement une estimation interne de l'erreur de généralisation.

Limites du bagging.

— **Réduction limitée du biais** : le bagging agit principalement sur la variance ; si les apprenants de base sont fortement biaisés (arbres très peu profonds, modèles linéaires pour des relations hautement non linéaires, etc.), l'ensemble restera sous-ajusté.

— **Faible gain en cas de forte corrélation** : lorsque les modèles de base sont très corrélés (données peu diversifiées, caractéristiques dominantes, absence de randomisation suffisante), la réduction de variance reste limitée alors que le coût computationnel (entraînement et inférence multipliés par m) devient important.

— **Petits jeux de données et OOB instable** : sur de très petits jeux de données, le bootstrap peut produire des échantillons très similaires, rendant l'estimation OOB instable et réduisant l'intérêt pratique de la méthode.

Boosting

Le boosting regroupe une famille de méthodes séquentielles où les modèles sont entraînés les uns après les autres, chacun se concentrant sur les erreurs commises par les précédents. Contrairement au bagging qui agit surtout sur la variance, le boosting cherche principalement à réduire le biais, en construisant progressivement un modèle additif puissant à partir de modèles de base faibles [12, 27, 4, 17].

Principe général. On considère un ensemble d'entraînement $\{(x_i, y_i)\}_{i=1}^n$ avec $y_i \in \{-1, +1\}$ pour fixer les idées. Le boosting maintient une distribution de poids $D_t(i)$ sur les exemples. À chaque itération t :

1. un *apprenant faible* h_t est entraîné sur les données pondérées par D_t ;
2. son erreur pondérée $\varepsilon_t = \sum_i D_t(i)\, \mathbb{I}\big(h_t(x_i) \neq y_i\big)$ est évaluée ;
3. un coefficient α_t mesurant la confiance accordée à h_t est calculé ;
4. les poids D_t sont mis à jour pour donner plus d'importance aux exemples mal prédits.

Le modèle final est une combinaison pondérée des h_t. Sous l'hypothèse de « faible apprentissage » (chaque h_t fait un peu mieux que le hasard), on peut montrer que l'erreur d'entraînement décroît de manière exponentielle avec le nombre d'itérations [12, 27].

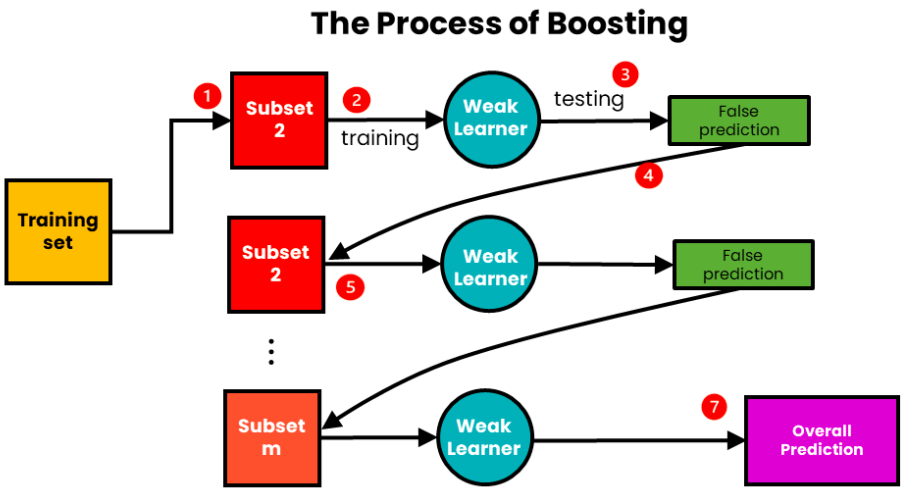


FIGURE 2.5 – Schéma pédagogique du boosting : chaîne de modèles où chaque étape corrige les erreurs de la précédente.

AdaBoost : formulation et réduction du biais. AdaBoost (Adaptive Boosting) est l'algorithme de boosting le plus classique [12, 27]. Sa version binaire peut se décrire comme suit.

1. **Initialisation.** On part d'une distribution uniforme

$$D_1(i) = \frac{1}{n}, \quad i = 1, \ldots, n.$$

2. **Pour** $t = 1, \ldots, T$ **:**

(a) on entraîne un modèle faible $h_t : \mathcal{X} \to \{-1, +1\}$ en minimisant l'erreur pondérée

$$\varepsilon_t = \sum_{i=1}^{n} D_t(i)\, \mathbb{I}\big(h_t(x_i) \neq y_i\big),$$

avec la contrainte $\varepsilon_t < 1/2$;

(b) on définit le poids du modèle

$$\alpha_t = \frac{1}{2} \log\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right);$$

(c) on met à jour les poids des exemples :

$$D_{t+1}(i) = \frac{D_t(i)\, \exp\big(-\alpha_t y_i h_t(x_i)\big)}{Z_t},$$

où Z_t est un facteur de normalisation assurant $\sum_i D_{t+1}(i) = 1$. Les exemples mal classés ($y_i h_t(x_i) < 0$) voient leur poids augmenter, ce qui force h_{t+1} à se focaliser sur eux.

Le classifieur final est

$$H(x) = \operatorname{sign}\left(\sum_{t=1}^{T} \alpha_t h_t(x)\right).$$

Théoriquement, on peut montrer que l'erreur d'entraînement de H est majorée par [27, 12]

$$\mathrm{err}_{\mathrm{train}}(H) \leq \prod_{t=1}^{T} Z_t \leq \exp\left(-2 \sum_{t=1}^{T} \gamma_t^2\right), \quad \text{où } \gamma_t = \tfrac{1}{2} - \varepsilon_t.$$

Si chaque apprenant faible est au moins un peu meilleur que le hasard ($\gamma_t \geq \gamma > 0$), l'erreur d'entraînement décroît donc comme $\exp(-2\gamma^2 T)$. En pratique, cela signifie que le biais du modèle global diminue rapidement avec le nombre d'itérations, tant que les apprenants restent effectivement faibles et diversifiés.

On peut aussi interpréter AdaBoost comme une méthode qui minimise de façon gloutonne la perte exponentielle

$$L(f) = \sum_{i=1}^{n} \exp\big(-y_i f(x_i)\big), \quad f(x) = \sum_{t=1}^{T} \alpha_t h_t(x),$$

par une descente de gradient coordonnée dans l'espace des fonctions. Cette vision explique pourquoi la marge $y_i f(x_i)$ joue un rôle central dans la performance [27].

En pratique, AdaBoost est souvent implémenté avec des *decision stumps* (arbres de profondeur 1) comme apprenants faibles ; dans scikit-learn, on peut l'utiliser via la classe `AdaBoostClassifier` avec un `DecisionTreeClassifier(max_depth=1)` comme estimateur de base. La complexité est en gros proportionnelle à T fois le coût d'entraînement d'un apprenant faible, et l'algorithme est naturellement itératif.

Gradient boosting : point de vue en espace de fonctions. Le gradient boosting généralise l'idée du boosting à toute fonction de perte différentiable [13]. On cherche à approximer une

fonction $F^\star$ en construisant un modèle additif

$$F_T(x) = \sum_{t=1}^{T} \nu\, \beta_t\, h_t(x),$$

où h_t sont des apprenants faibles (souvent des arbres de décision), β_t des coefficients et $\nu \in (0, 1]$ un facteur de « shrinkage » (taux d'apprentissage).

À chaque itération t :

1. on dispose d'un modèle courant F_{t-1} ;
2. on calcule, pour chaque exemple, les *pseudo-résidus* correspondant au gradient négatif de la perte par rapport aux prédictions :

$$r_{it} = -\left.\frac{\partial}{\partial F(x_i)} L\big(y_i, F(x_i)\big)\right|_{F=F_{t-1}} ;$$

   pour une perte quadratique, on obtient simplement $r_{it} = y_i - F_{t-1}(x_i)$;
3. on ajuste un apprenant faible h_t pour approximer ces pseudo-résidus, typiquement par régression aux moindres carrés :

$$h_t \approx \arg\min_h \sum_{i=1}^{n} \big(r_{it} - h(x_i)\big)^2;$$

4. on choisit un pas optimal β_t (par recherche de ligne) :

$$\beta_t = \arg\min_\beta \sum_{i=1}^{n} L\big(y_i, F_{t-1}(x_i) + \beta\, h_t(x_i)\big),$$

   puis on met à jour

$$F_t(x) = F_{t-1}(x) + \nu\, \beta_t\, h_t(x).$$

Pour la régression (perte quadratique), cela revient à ajuster successivement les résidus $y_i - F_{t-1}(x_i)$; pour la classification logistique, les pseudo-résidus correspondent à une erreur sur les probabilités estimées. Ainsi, chaque itération corrige systématiquement l'erreur résiduelle, ce qui diminue le biais tout en permettant un contrôle fin de la complexité via ν, la profondeur des arbres et le nombre d'itérations [13].

Des implémentations standards existent dans scikit-learn (`GradientBoostingClassifier`, `GradientBoostingRegressor`) ainsi que dans des bibliothèques dédiées comme XGBoost et LightGBM.

XGBoost et LightGBM : optimisations modernes. XGBoost [5] et LightGBM [21] sont des implémentations très optimisées du gradient boosting sur arbres de décision, qui ajoutent plusieurs idées clés :

— utilisation d'un développement de Taylor d'ordre 2 de la perte pour calculer, pour chaque feuille candidate, le gain associé à une scission en fonction de la somme des gradients G et des hessiennes H ;

— régularisation explicite des arbres via des termes de pénalisation L_2 sur les poids des feuilles et une pénalité sur le nombre de feuilles (paramètres λ et γ) ;

— sous-échantillonnage des observations et des variables (*row/column subsampling*) pour réduire la corrélation entre arbres et accélérer l'entraînement ;

— structures de données basées sur des histogrammes et croissance feuille-par-feuille (LightGBM) pour optimiser l'utilisation de la mémoire et du CPU.

Par exemple, dans XGBoost, le gain d'une scission est approximativement donné par [5]

$$\text{Gain} = \frac{1}{2}\left(\frac{G_L^2}{H_L+\lambda} + \frac{G_R^2}{H_R+\lambda} - \frac{(G_L+G_R)^2}{H_L+H_R+\lambda}\right) - \gamma,$$

où G_L, H_L (resp. G_R, H_R) sont les sommes des gradients et hessiennes dans les feuilles gauche et droite. Cette expression permet de choisir efficacement les meilleures scissions.

Du point de vue pratique, XGBoost et LightGBM sont devenus des standards de facto pour les problèmes tabulaires de grande dimension, grâce à leur excellent compromis entre biais (faible) et variance contrôlée par une régularisation riche et de nombreux hyperparamètres.

Limites du boosting.

— **Sensibilité au bruit et aux outliers** : en se focalisant sur les exemples les plus difficiles, le boosting (en particulier AdaBoost et certains gradient boosting peu régularisés) peut surajuster les observations bruitées ou mal étiquetées et dégrader la généralisation.

— **Risque de sur-apprentissage** : si l'on choisit des apprenants de base trop complexes ou un nombre d'itérations trop grand sans contrôle (pas d'*early stopping*, régularisation faible, taux d'apprentissage trop élevé), le biais devient très faible mais la variance augmente fortement.

— **Coût de calcul et complexité de réglage** : les implémentations modernes comme XGBoost et LightGBM atténuent ces effets via la pénalisation des arbres, le sous-échantillonnage et un réglage fin des hyperparamètres, au prix d'une complexité de configuration et d'un coût de calcul parfois élevés.

— **Interprétabilité limitée** : comme pour la plupart des ensembles d'arbres, l'interprétation globale du modèle reste difficile malgré l'existence d'indicateurs d'importance de variables et d'outils d'explicabilité locale.

Stacking (stacked generalization)

Le stacking, ou *stacked generalization*, est une méthode d'ensemble à deux niveaux où un modèle dit *méta-modèle* apprend comment combiner au mieux les prédictions de plusieurs modèles de base [35, 29]. Contrairement au vote ou à la simple moyenne, les poids de combinaison sont appris à partir des données, ce qui permet souvent de réduire l'erreur quadratique moyenne (MSE) en régression, ou l'erreur de classification, par rapport à chaque modèle pris isolément [3].

Architecture en couches. On distingue classiquement deux niveaux [35, 29] :

— **Niveau 0 (apprenants de base)** : plusieurs modèles potentiellement très différents (forêt aléatoire, gradient boosting, SVM, k-NN, réseaux neuronaux, etc.) sont entraînés sur les données d'entrée x pour prédire y ;

— **Niveau 1 (méta-modèle)** : un dernier modèle (souvent une régression linéaire pour la régression, ou une régression logistique pour la classification) prend en entrée les prédictions des modèles de niveau 0 et apprend une combinaison optimale.

En notation matricielle pour un problème de régression, si l'on dispose de K modèles de base produisant des prédictions $\hat{y}_k^{(i)}$ pour l'observation i, on forme la matrice de caractéristiques de niveau 1 :

$$Z = \begin{bmatrix} \hat{y}_1^{(1)} & \dots & \hat{y}_K^{(1)} \\ \vdots & & \vdots \\ \hat{y}_1^{(n)} & \dots & \hat{y}_K^{(n)} \end{bmatrix}, \quad y = \begin{bmatrix} y^{(1)} \\ \vdots \\ y^{(n)} \end{bmatrix}.$$

Le méta-modèle linéaire cherche alors des coefficients $w \in \mathbb{R}^K$ tels que

$$\hat{y}^{\text{stack}} = Zw$$

approche au mieux y au sens de la MSE. Le choix de w se fait en minimisant

$$\min_w \|y - Zw\|_2^2,$$

éventuellement sous la contrainte $w_k \geq 0$ comme proposé par Breiman [3]. Cette contrainte de non-négativité aide à garantir que le stacking ne dégrade pas les performances par rapport aux meilleurs modèles de base.

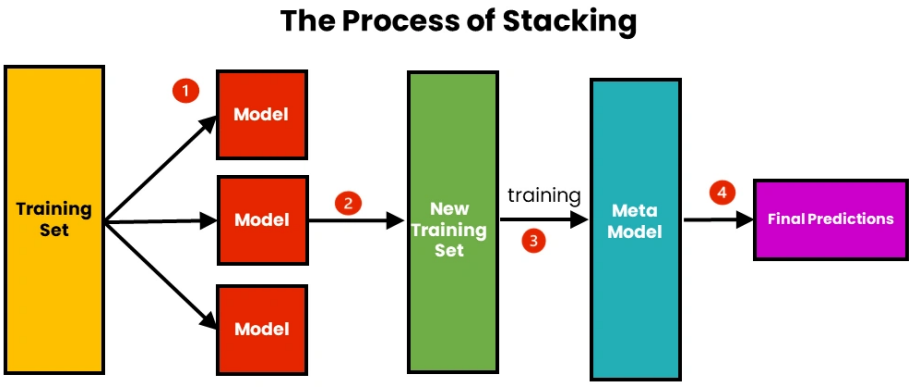


FIGURE 2.6 – Schéma pédagogique du stacking : architecture en couches avec apprenants de niveau 0 et méta-modèle de niveau 1.

Le problème du « data leakage ». Une implémentation naïve du stacking consisterait à entraîner les modèles de base sur tout le jeu d'entraînement D_{train} et à utiliser leurs prédictions sur ces mêmes exemples comme méta-caractéristiques :

$$Z_{ij}^{\text{naïf}} = L_i(x_j), \quad (x_j, y_j) \in D_{\text{train}},$$

où L_i désigne le i-ème modèle de base. Dans ce cas, le méta-modèle est ajusté sur des prédictions issues de données déjà vues par les modèles de niveau 0. On introduit ainsi une fuite d'information (*data leakage*) : les performances apparentes sur l'entraînement sont très optimistes, et le méta-

modèle apprend à exploiter des corrélations artificiellement fortes entre les sorties des modèles de base et la cible [29].

Pour obtenir une estimation non biaisée des sorties des modèles de base, il faut s'assurer que, pour chaque exemple (x_j, y_j), les prédictions utilisées au niveau 1 proviennent de modèles qui n'ont pas été entraînés sur (x_j, y_j). La solution est d'utiliser des prédictions *out-of-fold* (OOF) construites par validation croisée interne.

Etapes algorithmiques (stacking avec validation croisée). La procédure standard de stacking avec prédictions OOF se déroule comme suit :

1. **Sélection des modèles de base.** Le choix des apprenants de niveau 0 est crucial. On recherche :
   - **Diversité** : des modèles capturant des aspects différents du problème (modèles linéaires, arbres, réseaux de neurones, SVM, k-NN, etc.) ;
   - **Complémentarité** : des erreurs faiblement corrélées entre modèles, de sorte que chacun corrige les faiblesses des autres ;
   - **Qualité individuelle** : chaque modèle doit être raisonnablement performant pris isolément (un modèle très mauvais reste difficile à rattraper par le stacking) ;
   - **Hétérogénéité** : on privilégie des familles algorithmiques différentes plutôt que de nombreuses variantes très proches d'un même modèle.
2. **Découpage du jeu d'entraînement.** On partitionne le jeu de données en K plis de validation croisée $F_1, \ldots, F_K$.
3. **Entraînement des modèles de base et génération des méta-caractéristiques.** Pour chaque pli F_k :
   (a) on entraîne chaque modèle de base sur $D \setminus F_k$;
   (b) on applique ces modèles à F_k pour produire des prédictions $\hat{y}_k^{(i)}$ pour les exemples $i \in F_k$.

   En concaténant les prédictions sur tous les plis, on obtient, pour chaque exemple i, un vecteur $(\hat{y}_1^{(i)}, \ldots, \hat{y}_K^{(i)})$ obtenu sur des données *out-of-fold*.
4. **Apprentissage du méta-modèle.** On ajuste ensuite le méta-modèle sur la matrice Z et les vraies cibles y. Le choix du méta-modèle dépend du type de tâche et du compromis biais/variance recherché. Quelques options courantes sont :
   - **Régression linéaire / Ridge** : interprétable, stable, rapide ; bon choix par défaut en régression pour un premier essai ;
   - **Régression logistique** : produit des probabilités bien calibrées ; adaptée à la classification binaire ;
   - **Arbre de décision (profondeur modérée)** : permet de capturer des interactions non linéaires entre les sorties des modèles de base ;
   - **Gradient boosting peu profond** : très performant lorsque l'on dispose de beaucoup de données et que l'on souhaite modéliser des combinaisons complexes de prédicteurs de niveau 0 ;

— **Réseau de neurones peu profond** : utile pour des schémas de stacking plus profonds ou lorsque les relations entre modèles de base sont fortement non linéaires.

En pratique, on privilégie souvent des méta-modèles simples (régression linéaire ou logistique) pour limiter le risque de surajustement au niveau 1.

5. **Apprentissage final des modèles de base.** On réentraîne enfin tous les modèles de base sur la totalité du jeu d'entraînement, avant de les utiliser en test. En production, la prédiction d'un nouvel exemple $x^\star$ se fait en deux temps : (i) chaque modèle de base produit une prédiction ; (ii) le méta-modèle agrège ces prédictions.

Cette procédure garantit que le méta-modèle a vu des prédictions réalistes (obtenues sur des échantillons non vus par les modèles de base), évitant ainsi un surajustement massif.

Reduction du Biais et la Variance Le stacking agit à deux niveaux :

— **Réduction du biais**le méta-learner apprend à corriger les erreurs systématiques des base learners en apprenant une combinaison non-linéaire (si le méta est un arbre ou un réseau).

— **Réduction de la variance** en utilisant des prédictions croisées, il évite le surajustement et agit comme un régulariseur ; la variance finale est souvent inférieure à celle d'un simple vote ou moyenne.

Mathématiquement, si les erreurs des apprenants de base sont peu corrélées, la variance de la prédiction combinée est proche de

$$\mathrm{Var}\left(\sum_{i=1}^{K} w_i h_i(x)\right) \approx \sum_{i=1}^{K} w_i^2 \,\mathrm{Var}(h_i(x)),$$

où h_i désigne un modèle de base et w_i les poids appris par le méta-modèle. Dans le cas d'un simple stacking avec moyenne ($w_i = 1/K$), on retrouve

$$\mathrm{Var}_{\text{ensemble}} \approx \frac{1}{K} \sum_{i=1}^{K} \mathrm{Var}\big(h_i(x)\big)$$

Limites et risques du stacking. Malgré ses qualités, le stacking présente plusieurs limites qu'il faut garder à l'esprit :

— **Surapprentissage du méta-modèle** : si les lignes de Z sont trop corrélées à y (par exemple en cas de mauvais schéma OOF), le méta-modèle peut mémoriser des patterns propres au jeu d'entraînement plutôt que des relations généralisables.

— **Coût computationnel** : la génération des prédictions OOF nécessite d'entraîner chaque modèle de base sur plusieurs plis (environ $K \times F$ entraînements), puis d'entraîner le méta-modèle ; cela peut devenir coûteux pour de grands ensembles ou des modèles lourds.

— **Complexité de déploiement** : en production, il faut maintenir K modèles de base plus le méta-modèle, ce qui complique la mise à jour, la surveillance et l'observabilité.

— **Interprétabilité réduite** : le système forme une boîte noire à deux niveaux ; il est souvent difficile d'expliquer précisément pourquoi une décision a été prise.

— **Risque de leakage** : toute fuite d'information dans la génération des prédictions OOF (par exemple partage involontaire de données entre plis) invalide les hypothèses théoriques et peut conduire à un méta-modèle fortement biaisé.

Démonstration pratique : `StackingClassifier` sur Iris. Le code ci-dessous illustre une démonstration reproductible du stacking en classification sur le jeu de données Iris (trois classes, deux variables explicatives pour la visualisation). Trois apprenants de base hétérogènes sont utilisés : k-NN ($k = 1$), forêt aléatoire et classifieur bayésien naïf gaussien ; le méta-modèle est une régression logistique (`mlxtend.classifier.StackingClassifier`). Le premier bloc charge les données, le second construit le *StackingClassifier* et lance une validation croisée en 3 plis pour comparer l'exactitude de chaque modèle seul à celle de l'ensemble empilé : dans cette expérience, le stacking atteint la même précision que le meilleur base learner (forêt aléatoire, environ 95 %) tout en reposant sur une combinaison apprise des trois modèles. Un dernier bloc (optionnel) montre comment visualiser les régions de décision de chaque classifieur et de l'ensemble.

```
from sklearn import datasets

iris = datasets.load_iris()
X, y = iris.data[:, 1:3], iris.target
```

```
from sklearn import model_selection
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.ensemble import RandomForestClassifier
from mlxtend.classifier import StackingClassifier
import numpy as np
import warnings

warnings.simplefilter('ignore')

clf1 = KNeighborsClassifier(n_neighbors=1)
clf2 = RandomForestClassifier(random_state=1)
clf3 = GaussianNB()
lr = LogisticRegression()
sclf = StackingClassifier(classifiers=[clf1, clf2, clf3],
                          meta_classifier=lr)

print('3-fold cross validation:\n')

for clf, label in zip([clf1, clf2, clf3, sclf],
                      ['KNN',
                       'Random Forest',
```

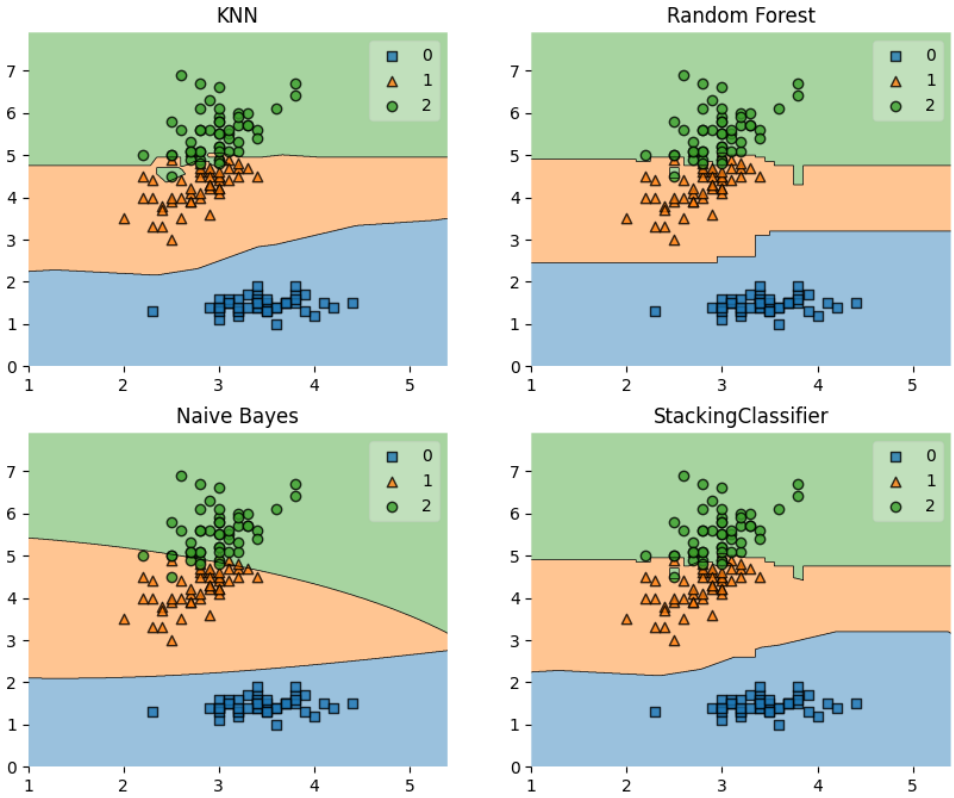
```
                          'Naive Bayes',
                          'StackingClassifier']):

    scores = model_selection.cross_val_score(clf, X, y,
                                             cv=3, scoring='
                                                accuracy')
    print("Accuracy: %0.2f (+/- %0.2f) [%s]"
          % (scores.mean(), scores.std(), label))
```

Output :

```
3-fold cross validation:

Accuracy: 0.91 (+/- 0.01) [KNN]
Accuracy: 0.95 (+/- 0.01) [Random Forest]
Accuracy: 0.91 (+/- 0.02) [Naive Bayes]
Accuracy: 0.95 (+/- 0.02) [StackingClassifier]
```

```
import matplotlib.pyplot as plt
from mlxtend.plotting import plot_decision_regions
import matplotlib.gridspec as gridspec
import itertools

gs = gridspec.GridSpec(2, 2)

fig = plt.figure(figsize=(10,8))

for clf, lab, grd in zip([clf1, clf2, clf3, sclf],
                         ['KNN',
                          'Random Forest',
                          'Naive Bayes',
                          'StackingClassifier'],
                          itertools.product([0, 1], repeat=2)):

    clf.fit(X, y)
    ax = plt.subplot(gs[grd[0], grd[1]])
    fig = plot_decision_regions(X=X, y=y, clf=clf)
    plt.title(lab)
```

Sortie :



Voting (vote majoritaire vs vote probabiliste)

Le vote est la méthode de fusion la plus simple pour les modèles parallèles [6, 15].

Hard voting (vote majoritaire). Chaque modèle donne une étiquette de classe. On choisit la classe qui a reçu le plus de suffrages. Par exemple :

— Modèle A → « chien »

— Modèle B → « chat »

— Modèle C → « chien »

La décision finale est alors « chien » (deux voix contre une).

Soft voting (vote probabiliste). On calcule la moyenne des probabilités prédites pour chaque classe. Cette méthode est plus précise car elle accorde plus d'importance aux modèles « sûrs d'eux ».

Considérons par exemple deux classes (chat, chien) et trois modèles :

— Modèle 1 : probabilités $[P(\text{chien}) = 0{,}4,\ P(\text{chat}) = 0{,}6] \Rightarrow$ prédit « chat »

— Modèle 2 : probabilités $[0{,}4,\ 0{,}6] \Rightarrow$ prédit « chat »

— Modèle 3 : probabilités $[0{,}9,\ 0{,}1] \Rightarrow$ prédit « chien »

En hard voting, la classe « chat » (deux voix) l'emporte. En soft voting, on calcule :

$$\overline{P}(\text{chien}) = \frac{0{,}4 + 0{,}4 + 0{,}9}{3} \approx 0{,}56, \qquad \overline{P}(\text{chat}) = \frac{0{,}6 + 0{,}6 + 0{,}1}{3} \approx 0{,}43.$$

La décision finale est alors « chien » : la forte conviction du modèle 3 compense l'incertitude des deux autres.

2.6 Analyse comparative des approches

Le choix d'une méthode d'ensemble dépend des contraintes du projet (volume de données, temps de réponse, précision requise, robustesse au bruit, coût de calcul, etc.).

Le tableau suivant résume quelques caractéristiques essentielles des trois grandes familles :

TABLE 2.2 – Comparaison synthétique bagging / boosting / stacking

Critère	Bagging	Boosting	Stacking
Objectif principal	Réduire la variance	Réduire le biais	Maximiser la précision
Type de modèles	Homogènes (arbres profonds)	Homogènes (arbres peu profonds)	Hétérogènes (divers types)
Entraînement	Parallèle (indépendant)	Séquentiel (dépendant)	Parallèle + méta-apprentissage
Sensibilité au bruit	Faible (robuste)	Élevée (risque de sur-ajuster)	Variable
Risque d'overfitting	Faible	Élevé (si non régularisé)	Moyen (si données limitées)
Coût de calcul	Efficace (parallélisable)	Lent (entraînement en série)	Très élevé

2.7 Avantages et limites de la combinaison

Avantages

1. **Précision supérieure** : les ensembles dépassent quasi systématiquement les modèles uniques en combinant des perspectives complémentaires.
2. **Généralisation et robustesse** : en lissant les prédictions, les systèmes MCS sont moins sensibles aux anomalies statistiques d'un seul jeu de données.
3. **Traitement des données complexes** : ils permettent de diviser un problème complexe en sous-tâches traitées par des experts dédiés (*divide and conquer*).

Limites

1. **Coût en ressources** : l'entraînement de dizaines ou de centaines de modèles consomme du temps CPU, de la mémoire et de l'énergie.
2. **Opacité (« black box »)** : il est extrêmement difficile d'expliquer pourquoi un ensemble de plusieurs centaines d'arbres de décision a pris une décision spécifique.
3. **Complexité de maintenance** : gérer un pipeline avec plusieurs modèles hétérogènes demande une infrastructure plus lourde qu'un modèle monolithique.

2.8 Conclusion

La combinaison de classifieurs représente une approche mature et extrêmement puissante de l'apprentissage automatique. En orchestrant judicieusement la diversité et en s'appuyant sur des fondements statistiques rigoureux, les systèmes multi-classifieurs permettent de transformer une collection de modèles imparfaits en un outil de décision d'une fiabilité exceptionnelle [20, 31, 14, 11].

3 Explicabilité des modèles

3.1 Introduction

L'essor des modèles d'apprentissage automatique complexes, en particulier les modèles non linéaires à haute capacité de représentation, a profondément transformé les systèmes de décision automatisés. Les architectures modernes, telles que les réseaux de neurones profonds, les forêts aléatoires, les méthodes de boosting et les ensembles de modèles, permettent d'atteindre des niveaux de performance prédictive élevés, mais au prix d'une perte progressive de transparence et de lisibilité du raisonnement algorithmique.

Cette opacité structurelle engendre ce que l'on qualifie de problème de la *boîte noire algorithmique*, dans lequel la performance prédictive ne s'accompagne plus d'une compréhension explicite des mécanismes décisionnels. Or, dans de nombreux contextes critiques (santé, finance, justice, sécurité, gouvernance publique), la validité d'une décision ne repose pas uniquement sur sa précision statistique, mais également sur sa compréhensibilité, sa justifiabilité et sa traçabilité.

L'explicabilité des modèles émerge ainsi comme un champ scientifique visant à établir une correspondance intelligible entre les données d'entrée, les transformations internes du modèle et la décision finale.

Notre cas d'étude. Tout au long de ce chapitre, nous utiliserons un exemple concret : le déploiement d'un modèle pour le **diagnostic précoce de la maladie rénale chronique** (CKD — *Chronic Kidney Disease*). Le jeu de données de référence (UCI Machine Learning Repository) contient **400 patients** décrits par **24 variables médicales** : taux d'hémoglobine, pression artérielle, gravité spécifique de l'urine, présence de diabète, etc. La variable cible est binaire : CKD présente ou absente. Ce contexte médical illustre parfaitement l'enjeu : un modèle ultra-précis mais inexplicable est inutilisable par un médecin, car une décision thérapeutique engage la vie d'un patient.

3.2 Définition de l'explicabilité

Définition (Modèle). Un modèle d'apprentissage automatique est défini comme une fonction :

$$f : X \to Y$$

où :

— $X \subseteq \mathbb{R}^n$ est l'espace des variables d'entrée,

— Y est l'espace des sorties (classes ou valeurs continues),

— $x \in X$ une observation,

— $\hat{y} = f(x)$ la prédiction du modèle.

Dans les modèles complexes, la structure interne de la fonction f est non linéaire, distribuée et multidimensionnelle, ce qui rend la relation entre x et $\hat{y}$ non directement intelligible pour un observateur humain.

Formulation conceptuelle

L'explicabilité ne cherche pas à modifier le modèle f, mais à construire une projection interprétable de son comportement décisionnel. Elle agit comme une transformation :

$$f \longrightarrow h$$

où h est une représentation compréhensible par l'humain du fonctionnement de f.

Définition (Explicabilité). *La capacité d'un système d'apprentissage automatique à produire une représentation intelligible, formelle et interprétable du lien fonctionnel entre les données d'entrée, les mécanismes internes du modèle et la décision produite.*

Sur le plan mathématique, elle repose sur l'existence d'une fonction d'explication :

$$E : (f, x) \to H$$

telle que E transforme un modèle opaque en une représentation compréhensible pour un utilisateur humain, où H est un espace de représentations compréhensibles par l'humain (règles, poids, scores d'influence, graphes, visualisations, relations causales).

Exemple (Explicabilité simple). Considérons un modèle de classification :

$$f(x_1, x_2, x_3) = \hat{y}$$

Une fonction d'explication peut produire un vecteur de contributions :

$$E(f, x) = (\alpha_1, \alpha_2, \alpha_3)$$

avec :

$$\sum_{i=1}^{3} \alpha_i = 1$$

où α_i représente la contribution relative de la variable x_i dans la décision finale.

Interprétation :

— si $\alpha_1 = 0.6$, alors la variable x_1 explique 60% de la décision,

— si $\alpha_2 = 0.3$, elle en explique 30%,

— si $\alpha_3 = 0.1$, elle en explique 10%.

Positionnement systémique

L'explicabilité introduit une nouvelle couche fonctionnelle dans les systèmes d'IA :

$$\text{Données} \longrightarrow \text{Modèle} \longrightarrow \text{Décision} \longrightarrow \text{Explication}$$

Elle transforme ainsi un système prédictif en un système décisionnel explicable, où la décision n'est plus seulement produite, mais également justifiée, traçable et compréhensible.

3.3 Interprétabilité vs explicabilité

Bien que souvent utilisées de manière interchangeable, les notions d'interprétabilité et d'explicabilité renvoient à deux concepts théoriquement distincts.

Définition (Interprétabilité). L'interprétabilité désigne la capacité intrinsèque d'un modèle à être compris directement par un humain, sans mécanisme externe d'explication. Elle est une propriété structurelle du modèle lui-même.

Formellement, un modèle interprétable est une fonction $f : X \to Y$ telle que sa structure interne $S(f)$ soit directement lisible et compréhensible :

$$S(f) \in H$$

où H est un espace de représentations humaines interprétables (équations simples, règles logiques, arbres, coefficients explicites).

Exemple (Modèle linéaire interprétable). Modèle linéaire :

$$f(x) = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Chaque coefficient w_i représente directement l'influence de la variable x_i, ce qui rend le modèle interprétable par construction.

Définition (Explicabilité (rappel)). L'explicabilité désigne la capacité à expliquer un modèle, y compris lorsqu'il est intrinsèquement opaque. Elle est une propriété fonctionnelle externe au modèle, reposant sur une fonction d'explication $E : (f, x) \to H$.

Différence structurelle fondamentale

Table 3.1 – Différences clés entre interprétabilité et explicabilité

Dimension	**Interprétabilité**	**Explicabilité**
Nature	Propriété du modèle	Propriété du système
Position	Interne	Externe
Structure	Transparente	Opaque
Explication	Native	Post-hoc
Dépendance à des outils	Non	Oui
Complexité admissible	Faible	Élevée

Typologie formelle des modèles

1. **Modèles interprétables et explicables** : $f \in I \cap E$
   $\rightarrow$ modèles simples (linéaires, règles, arbres simples)
2. **Modèles non interprétables mais explicables** : $f \notin I, f \in E$
   $\rightarrow$ réseaux de neurones, ensembles complexes + méthodes XAI
3. **Modèles non interprétables et non explicables** : $f \notin I, f \notin E$
   $\rightarrow$ systèmes opaques non auditables

Exemple (Comparatif CKD). **Modèle interprétable :**

$$f(x) = 2 \times \text{hémoglobine} - 0.5 \times \text{gravité spécifique}$$

$\rightarrow$ lecture directe de l'influence des variables.

Modèle non interprétable :

$$f(x) = \sigma(W_3 \cdot \sigma(W_2 \cdot \sigma(W_1 x)))$$

$\rightarrow$ réseau neuronal profond, structure non lisible.

Explication possible :

$$E(f, x) = (\alpha_{\text{hémoglobine}}, \alpha_{\text{gravité}}, \dots)$$

$\rightarrow$ contributions explicatives post-hoc.

Point clé. L'interprétabilité est une propriété intrinsèque d'un modèle liée à sa structure interne, tandis que l'explicabilité est une propriété extrinsèque liée à la capacité du système à produire des représentations compréhensibles de son comportement décisionnel.

Conséquence scientifique majeure

Cette distinction implique que l'on peut optimiser :

— soit la simplicité structurelle du modèle (interprétabilité),

— soit la performance prédictive du modèle, compensée par des mécanismes d'explication (explicabilité),

— soit un compromis hybride entre les deux.

Formalisation du compromis :

$$\max_f P(f) \quad \text{sous contrainte} \quad C(f)$$

où :

— $P(f)$ = performance prédictive,

— $C(f)$ = compréhensibilité humaine.

3.4 Méthodes d'explicabilité

Les méthodes d'explicabilité regroupent l'ensemble des techniques permettant de produire des représentations compréhensibles du comportement décisionnel d'un modèle d'apprentissage automatique. Formellement, étant donné un modèle $f : X \to Y$ et une observation $x \in X$, les méthodes d'explicabilité cherchent à construire une fonction d'explication $E : (f, x) \to H$.

Modèles intrinsèquement interprétables

Les modèles intrinsèquement interprétables sont des modèles dont la structure interne est directement compréhensible par un humain, sans nécessiter de mécanisme externe d'explication.

1. Modèles linéaires

$$f(x) = w^\top x + b = \sum_{i=1}^{n} w_i x_i + b$$

Interprétation directe : $\frac{\partial f}{\partial x_i} = w_i$

2. Régression logistique

$$P(y = 1 \mid x) = \sigma(w^\top x + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Les coefficients w_i sont interprétables comme des log-odds ratios.

3. Arbres de décision Un arbre de décision est un ensemble hiérarchique de règles :

$$\text{si } x_1 < \theta_1 \text{ alors } \begin{cases} \text{si } x_2 < \theta_2 \to y_1 \\ \text{sinon} \to y_2 \end{cases}$$

4. Systèmes de règles

$$R = \{r_1, r_2, \ldots, r_m\}, \quad r_k : \text{SI } C_k(x) \text{ ALORS } y_k$$

5. Modèles additifs généralisés (GAM)

$$f(x) = \beta_0 + \sum_{i=1}^{n} f_i(x_i)$$

Point clé. Un modèle intrinsèquement interprétable est un modèle dont la structure mathématique permet une lecture directe du raisonnement décisionnel, sans mécanisme externe d'explication.

Méthodes post-hoc

Les méthodes post-hoc désignent l'ensemble des techniques d'explicabilité appliquées après l'entraînement d'un modèle, sans en modifier la structure interne.

1. Feature Importance La *feature importance* construit un vecteur :

$$I(x) = (I_1, I_2, \ldots, I_n), \quad I_i \in \mathbb{R}$$

où I_i représente l'importance de la variable x_i dans la décision $f(x)$.

Principes mathématiques fondamentaux :

1. **Importance par sensibilité (gradient)** : $I_i = \left|\frac{\partial f(x)}{\partial x_i}\right|$
2. **Importance par perturbation** : $I_i = |f(x) - f(x^{(i)})|$
3. **Importance par information** : $I_i \propto \Delta L(f)$

Définition (Feature Importance). La *feature importance* est une fonction d'attribution I qui projette une prédiction complexe $f(x)$ dans un espace explicatif vectoriel, où chaque composante quantifie l'influence relative d'une variable d'entrée sur la décision du modèle :

$$I : (f, x) \to \mathbb{R}^n$$

2. LIME (Local Interpretable Model-Agnostic Explanations) LIME est une méthode d'explicabilité post-hoc, locale et agnostique au modèle, qui explique une prédiction individuelle en construisant un modèle interprétable localement équivalent.

Formulation mathématique :

$$\arg\min_{g \in G} L(f, g, \pi_x) + \Omega(g)$$

où :

— f : modèle complexe (boîte noire)

— g : modèle interprétable (régression linéaire, arbre simple)

— L : fonction de perte mesurant la différence entre f et g

— π_x : fonction de proximité locale $\pi_x(z) = \exp\left(-\frac{D(x,z)^2}{2\sigma^2}\right)$

— $\Omega(g)$: pénalité de complexité du modèle explicatif

LIME appliqué au diagnostic CKD. Pour un patient avec un taux d'hémoglobine de 9.2 g/dL et une gravité spécifique de 1.010, LIME génère des perturbations locales et apprend un modèle linéaire :

$$g(x) = 0.75 \times \text{hémoglobine} - 0.45 \times \text{gravité} + 0.10 \times \text{âge}$$

Interprétation : l'hémoglobine contribue fortement positivement au risque CKD, la gravité spécifique contribue négativement (plus elle est basse, plus le risque est élevé), l'âge a un effet mineur.

Attention. L'instabilité de LIME est un problème critique en contexte médical. Si deux applications successives sur le même patient donnent des importances contradictoires, le médecin ne peut pas faire confiance à l'explication.

3. SHAP (SHapley Additive exPlanations) SHAP est une méthode d'explicabilité post-hoc fondée sur la théorie des jeux coopératifs. Chaque variable est considérée comme un joueur contribuant à la prédiction finale.

Principe général : pour une observation $x = (x_1, x_2, \ldots, x_n)$, SHAP cherche une décomposition additive :

$$f(x) = \phi_0 + \sum_{i=1}^{n} \phi_i$$

où ϕ_0 est la valeur de base (prédiction moyenne) et ϕ_i la contribution de la variable x_i.

Fondement théorique (valeur de Shapley) :

$$\phi_i(f, x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} \big[f(S \cup \{i\}) - f(S) \big]$$

Point clé. SHAP est la seule méthode qui satisfait simultanément les propriétés d'**efficacité**, de **symétrie**, de **nullité** et d'**additivité**.

Pour les ensembles d'arbres, **TreeSHAP** permet un calcul efficace en temps polynomial :

$$\mathcal{O}(2^M) \longrightarrow \mathcal{O}(T \cdot L \cdot D^2)$$

où T = nombre d'arbres, L = feuilles par arbre, D = profondeur maximale.

TreeSHAP appliqué au diagnostic CKD. Un nouveau patient est admis avec un score de risque de 88%. TreeSHAP retourne :

Variable	ϕ_i (valeur SHAP)	Interprétation
Taux d'hémoglobine (faible)	+0.42	Pousse fortement vers CKD
Gravité spécifique urine (basse)	+0.31	Pousse fortement vers CKD
Présence de diabète mellitus	+0.15	Pousse vers CKD
Pression artérielle (normale)	−0.08	Réduit légèrement le risque
Âge du patient (50 ans)	+0.03	Impact marginal
Score moyen du modèle	+0.05	Valeur de référence

Somme des ϕ_i + score moyen = 0.88 (88%)

Le médecin voit immédiatement que ce sont principalement le faible taux d'hémoglobine et la gravité spécifique anormale qui drive la prédiction — des marqueurs physiologiquement cohérents avec la CKD.

4. PDP & ICE Plots Les PDP (Partial Dependence Plots) et ICE (Individual Conditional Expectation) sont des méthodes basées sur l'analyse fonctionnelle de la relation entre une variable et la prédiction.

Partial Dependence Plot (PDP) :

$$PD(x_s) = \mathbb{E}_{X_c}\big[f(x_s, X_c)\big] \approx \frac{1}{N}\sum_{i=1}^{N} f(x_s, x_c^{(i)})$$

Individual Conditional Expectation (ICE) :

$$ICE_i(x_s) = f(x_s, x_c^{(i)})$$

Relation : $PD(x_s) = \frac{1}{N}\sum_{i=1}^{N} ICE_i(x_s)$

PDP et ICE pour la CKD.

— PDP : montre qu'en moyenne, le risque CKD diminue quand l'hémoglobine augmente.

— ICE : révèle que cette relation est plus marquée chez les patients diabétiques que chez les non-diabétiques, indiquant une interaction entre hémoglobine et diabète.

3.5 Explicabilité locale vs globale

L'explicabilité peut être analysée selon deux niveaux complémentaires : locale (centrée sur les décisions individuelles) et globale (orientée vers la compréhension du comportement général du modèle).

Explicabilité locale

L'explicabilité locale vise à expliquer une prédiction spécifique $\hat{y} = f(x)$ en cherchant une fonction explicative locale g_x telle que :

$$\forall z \in N(x), \quad g_x(z) \approx f(z), \quad N(x) = \{z \in X \mid D(x, z) \leq \epsilon\}$$

Objectifs :

— Justification d'une décision individuelle

— Explicabilité contextuelle

— Auditabilité locale

— Acceptabilité utilisateur

Méthodes associées : LIME, SHAP (local), ICE, contre-factuels.

Explicabilité globale

L'explicabilité globale vise à comprendre la structure décisionnelle du modèle sur l'ensemble du domaine :

$$\mathbb{E}_{x\sim P(X)}[\|f(x) - G(x)\|] \leq \delta$$

Objectifs :

— Compréhension structurelle

— Audit du modèle

— Validation scientifique

— Gouvernance algorithmique

Méthodes associées : SHAP (global), PDP, feature importance globale, règles globales.

Différences structurelles

TABLE 3.2 – Comparaison explicabilité locale vs globale

Dimension	**Locale**	**Globale**
Portée	individuelle	populationnelle
Espace	voisinage $N(x)$	espace X
Objectif	justification	compréhension
Usage	décision	gouvernance
Temporalité	instantanée	structurelle
Nature	contextuelle	systémique

Point clé. L'explicabilité complète = explicabilité locale $\oplus$ explicabilité globale. Aucune ne suffit seule. Un système explicable doit satisfaire : $\forall x \in X, \exists g_x$ (local) et $\exists G$ (global).

3.6 Conclusion — Importance décisionnelle, enjeux éthiques et réglementaires

L'explicabilité s'impose aujourd'hui comme une condition structurelle du fonctionnement des systèmes décisionnels fondés sur l'intelligence artificielle. Elle ne constitue plus un simple outil d'analyse, mais un mécanisme de **gouvernance algorithmique**, garantissant la **traçabilité**, la **justification**, l'**auditabilité** et la **responsabilité** des décisions automatisées.

Points clés du chapitre :

1. **Définition formelle** : L'explicabilité est la capacité à produire une représentation intelligible du lien entre entrées et sortie d'un modèle, via une fonction d'explication $E : (f, x) \to H$.
2. **Interprétabilité vs explicabilité** : L'interprétabilité est une propriété intrinsèque du modèle ; l'explicabilité est une propriété extrinsèque du système. Cette distinction est fondamentale pour comprendre les compromis de conception.
3. **Méthodes d'explicabilité** : Les modèles intrinsèquement interprétables (linéaires, arbres, GAM) offrent une transparence native mais une expressivité limitée. Les méthodes post-hoc (feature importance, LIME, SHAP, PDP/ICE) permettent d'expliquer des modèles complexes sans en modifier la structure.
4. **SHAP et TreeSHAP** : Fondées sur la théorie des jeux, ces méthodes offrent des garanties mathématiques uniques d'équité, de cohérence et d'additivité, avec une implémentation efficace pour les ensembles d'arbres.

5. **Locale vs globale** : L'explicabilité locale justifie les décisions individuelles ; l'explicabilité globale permet de comprendre le comportement général du modèle. Les deux niveaux sont complémentaires et nécessaires.

Enjeux pour le cas CKD. Dans un contexte médical, l'explicabilité n'est pas optionnelle. Elle conditionne :

— **La confiance clinique** : un médecin ne peut pas traiter sur la base d'une boîte noire.

— **La détection de biais** : seule l'explicabilité permet de vérifier que le modèle s'appuie sur des biomarqueurs pertinents (hémoglobine, créatinine) et non sur des artefacts.

— **La conformité légale** : RGPD (droit à l'explication), normes FDA, certifications CE exigent la traçabilité des décisions automatisées.

— **L'acceptabilité sociale** : les patients ont le droit de comprendre pourquoi une décision thérapeutique est prise.

Parallèlement, l'explicabilité devient un enjeu **éthique** et **réglementaire** majeur. Elle conditionne le respect des principes fondamentaux de **justice**, d'**équité**, de **non-discrimination** et de **responsabilité morale**, tout en répondant aux exigences juridiques de **transparence**, de **droit à l'explication** et de **responsabilité légale**. Ainsi, l'explicabilité ne relève plus uniquement de l'ingénierie des modèles, mais s'inscrit dans une **vision systémique de l'IA**, où performance algorithmique, acceptabilité sociale, conformité réglementaire et responsabilité institutionnelle doivent être conjointement intégrées. Elle devient, de ce fait, un **pilier fondamental** de la conception des systèmes d'intelligence artificielle **durables, fiables et légitimes**.

4 Combinaison de Classifieurs et Explicabilité

> *« Un modèle qui prédit bien mais que personne ne comprend est aussi dangereux qu'un modèle qui ne prédit pas du tout. »*

Ce chapitre aborde une tension centrale de l'apprentissage automatique moderne : les méthodes les plus performantes — appelées *ensembles de classifieurs* ou *méthodes d'ensemble* — sont précisément les plus difficiles à interpréter. Nous commencerons par comprendre pourquoi le regroupement de plusieurs modèles crée un problème dit de « boîte noire », puis nous analyserons les sources mathématiques de cette complexité, avant d'explorer en détail les stratégies permettant de réconcilier performance et transparence.

4.1 Le Problème de la Boîte Noire dans les Ensembles

Qu'est-ce qu'une boîte noire en apprentissage automatique ?

Un modèle de machine learning est dit **boîte noire** (black box) lorsque sa structure interne est trop complexe pour qu'un être humain puisse suivre mentalement le chemin de la décision. On lui fournit des données en entrée, on obtient une prédiction en sortie, mais le raisonnement intermédiaire reste opaque.

Définition (Boîte noire (Black Box)). Un modèle $f : \mathcal{X} \to \mathcal{Y}$ est qualifié de **boîte noire** lorsqu'il est impossible, en pratique, pour un observateur humain de reconstituer le processus de décision complet à partir de la représentation interne du modèle, même en ayant accès à tous ses paramètres.

L'opposition classique se fait entre les modèles *interprétables par conception* (un arbre de décision de profondeur 3, une régression linéaire) et les modèles *opaques par nature* (un réseau de neurones profond, une forêt aléatoire de 500 arbres).

Pourquoi les ensembles de classifieurs deviennent-ils des boîtes noires ?

Imaginons un étudiant qui doit décider si une image médicale montre une tumeur bénigne ou maligne. S'il consulte un seul médecin, il peut suivre son raisonnement pas à pas. S'il consulte simultanément 1 000 médecins et combine leurs avis selon une règle mathématique complexe, il devient impossible de retracer la logique de la décision finale.

C'est exactement ce qui se passe avec les méthodes d'ensemble. Elles combinent plusieurs modèles — appelés *classifieurs de base* ou *apprenants faibles* — pour produire une prédiction plus robuste. Il existe trois grandes familles :

1. **Bagging (Bootstrap Aggregating) :** les modèles sont entraînés en parallèle sur des sous-échantillons aléatoires du jeu de données d'entraînement. Exemple canonique : la *Forêt Aléatoire* (Random Forest).
2. **Boosting :** les modèles sont entraînés séquentiellement, chaque nouveau modèle cherchant à corriger les erreurs du précédent. Exemples : AdaBoost, Gradient Boosting, XGBoost, LightGBM.
3. **Stacking (Stacked Generalization) :** les prédictions des modèles de base sont utilisées

comme entrées d'un modèle de second niveau (*méta-apprenant*) qui apprend à les combiner de façon optimale.

Remarque (Pourquoi l'opacité est-elle dangereuse ?). Dans des domaines critiques comme la médecine (diagnostic de cancer), la finance (octroi de crédit) ou la justice (aide à la décision judiciaire), un modèle opaque est problématique car : (1) il est impossible d'identifier et corriger des biais discriminatoires ; (2) les réglementations (comme le RGPD en Europe) exigent un « droit à l'explication » ; (3) les experts métiers ne peuvent pas valider la cohérence du raisonnement.

L'effet cascade de l'agrégation

La décision finale d'un ensemble de M classifieurs $\{h_1, h_2, \ldots, h_M\}$ pour un vote majoritaire simple s'écrit :

$$\hat{y}(\mathbf{x}) = \arg\max_{c \in \mathcal{C}} \sum_{m=1}^{M} \mathbf{1}\big[h_m(\mathbf{x}) = c\big] \tag{13}$$

où $\mathbf{1}[\cdot]$ est la fonction indicatrice et $\mathcal{C}$ l'ensemble des classes.

Dans le cas continu (agrégation de probabilités), on obtient :

$$\hat{y}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^{M} h_m(\mathbf{x}) \tag{14}$$

Si l'on cherche à expliquer pourquoi $\hat{y}(\mathbf{x}_0) = 1$ pour une instance particulière $\mathbf{x}_0$, il faut comprendre pourquoi *chacun* des M classifieurs a voté dans ce sens, et comment leurs votes individuels ont été agrégés. Avec $M = 500$ arbres ayant chacun des centaines de nœuds de décision, cette tâche devient humainement impossible sans outils algorithmiques dédiés.

4.2 Complexité des Ensembles de Modèles

La dimension de Vapnik-Chervonenkis et l'explosion de la capacité

Pour mesurer formellement pourquoi les ensembles sont si difficiles à interpréter, il faut comprendre la notion de **dimension de Vapnik-Chervonenkis** (dimension VC), qui quantifie la richesse de l'espace des hypothèses d'un algorithme.

Définition (Dimension de Vapnik-Chervonenkis). Soit $\mathcal{H}$ un espace d'hypothèses (ensemble de fonctions de classification). La **dimension VC** de $\mathcal{H}$, notée $\mathrm{VC}(\mathcal{H})$, est le plus grand entier d tel qu'il existe un ensemble de d points $\{\mathbf{x}_1, \ldots, \mathbf{x}_d\} \subset \mathcal{X}$ que $\mathcal{H}$ peut *pulvériser*, c'est-à-dire réaliser toutes les 2^d affectations binaires possibles sur ces points.

Exemple concret : Un arbre de décision de profondeur maximale k sur p variables binaires possède une dimension VC de l'ordre de $O(k \cdot p)$. C'est une valeur finie et contrôlée. En revanche, une forêt aléatoire de M arbres de profondeur k génère une frontière de décision dont la complexité combinatoire est de l'ordre de :

$$\mathrm{VC}(\mathcal{H}_{\text{forêt}}) = O(M \cdot k \cdot p \cdot \log(M)) \tag{15}$$

Cette explosion de la dimension VC a deux conséquences directes sur l'interprétabilité :

1. **Perte de simulabilité :** Un humain ne peut plus simuler mentalement le processus décisionnel. Il faut au minimum $O(\mathrm{VC}(\mathcal{H}))$ exemples pour comprendre le comportement du modèle.
2. **Perte de décomposabilité :** Il devient impossible d'attribuer un coefficient fixe et indépendant à chaque variable d'entrée. La contribution d'une variable x_j dépend des valeurs de toutes les autres variables, de façon non linéaire et locale.

La décomposition biais-variance et le compromis d'opacité

La théorie statistique de l'apprentissage décompose l'erreur attendue d'un modèle h en trois termes :

$$\mathbb{E}\big[(y - h(\mathbf{x}))^2\big] = \underbrace{\mathrm{Biais}^2[h(\mathbf{x})]}_{\text{erreur systématique}} + \underbrace{\mathrm{Variance}[h(\mathbf{x})]}_{\text{instabilité}} + \underbrace{\sigma_\epsilon^2}_{\text{bruit irréductible}} \tag{16}$$

Les méthodes d'ensemble exploitent cette décomposition de manière différente selon leur type :

— Le **Bagging** réduit principalement la *variance* : plusieurs modèles entraînés sur des sous-échantillons différents produisent des erreurs décorrélées, dont la moyenne est plus stable.

— Le **Boosting** réduit principalement le *biais* : les modèles successifs corrigent les erreurs résiduelles accumulées.

Démonstration de la réduction de variance par le Bagging :

Soit M classifieurs de base $h_1, \ldots, h_M$, entraînés sur des sous-ensembles bootstrap indépendants, chacun ayant une variance σ^2 et une covariance pairwise $\rho\sigma^2$ (avec ρ le coefficient de corrélation entre deux prédicteurs). La variance de leur moyenne est :

$$\mathrm{Var}\left(\frac{1}{M}\sum_{m=1}^{M} h_m(\mathbf{x})\right) = \rho\sigma^2 + \frac{1-\rho}{M}\,\sigma^2 \tag{17}$$

Lorsque $M \to \infty$, le second terme tend vers zéro et la variance résiduelle est $\rho\sigma^2$: elle est entièrement déterminée par la **corrélation entre les classifieurs**. C'est pourquoi les Random Forests introduisent la sélection aléatoire de variables : pour maximiser la diversité (réduire ρ).

Remarque (Paradoxe de l'opacité). La diversité entre classifieurs est *à la fois* la cause de la performance supérieure *et* la cause principale de l'opacité. En forçant les modèles à raisonner sur des sous-espaces de variables différents, on rend impossible toute explication unifiée et cohérente de la décision finale.

Le théorème de Condorcet et ses implications pour l'explicabilité

Le **théorème du jury de Condorcet** (1785) fournit la justification théorique de l'agrégation par vote majoritaire :

Théorème (Jury de Condorcet). Supposons M votants indépendants, chacun prenant la bonne décision avec probabilité $p > 0.5$. La probabilité que la majorité prenne la bonne décision

est :

$$P_M = \sum_{k=\lceil M/2 \rceil}^{M} \binom{M}{k} p^k (1-p)^{M-k} \tag{18}$$

et $P_M \to 1$ lorsque $M \to \infty$.

Ce que cela implique pour l'explicabilité : La force de l'ensemble réside précisément dans la *faiblesse et la spécialisation locale* de chaque apprenant de base. Un apprenant performant sur une sous-région de l'espace n'a pas besoin d'être correct ailleurs : les autres compensent. Cela signifie qu'il est vain de chercher à expliquer la décision finale par un raisonnement global cohérent : la décision émerge de la *combinaison de raisonnements locaux et partiellement contradictoires.*

L'identité de Krogh-Vedelsby : diversité et erreur d'ensemble

L'identité de Krogh et Vedelsby (1995) formalise mathématiquement le rôle de la diversité dans la qualité d'un ensemble. Soit $\bar{h}(\mathbf{x}) = \frac{1}{M}\sum_{m=1}^{M} h_m(\mathbf{x})$ l'agrégat des prédictions, l'erreur quadratique moyenne de l'ensemble vérifie :

$$\mathbb{E}_{\mathbf{x}}\big[(y-\bar{h}(\mathbf{x}))^2\big] = \underbrace{\frac{1}{M}\sum_{m=1}^{M}\mathbb{E}\big[(y-h_m(\mathbf{x}))^2\big]}_{\text{erreur moyenne des classifieurs}} - \underbrace{\frac{1}{M}\sum_{m=1}^{M}\mathbb{E}\big[(\bar{h}(\mathbf{x})-h_m(\mathbf{x}))^2\big]}_{\text{ambiguïté (diversité)}} \tag{19}$$

Interprétation : L'erreur de l'ensemble est *toujours strictement inférieure* à la moyenne des erreurs individuelles, l'écart étant exactement la diversité des prédictions. Plus les classifieurs désaccordent sur leurs prédictions (grande diversité), plus l'ensemble gagne en précision. Mais ce désaccord est précisément ce qui rend impossible toute explication unifiée : des classifieurs qui s'accordent sont interprétables ; des classifieurs qui se contredisent délibérément ne le sont pas.

L'accumulation des résidus dans le Boosting

Le Gradient Boosting construit son modèle de façon additive et séquentielle :

$$F_T(\mathbf{x}) = \sum_{t=1}^{T} \eta \cdot h_t(\mathbf{x}) \tag{20}$$

où η est le taux d'apprentissage et h_t est le t-ième arbre entraîné sur le gradient de la perte à l'itération t :

$$h_t = \arg\min_{h\in\mathcal{H}} \sum_{i=1}^{n} \left[-g_i^{(t)}\right] \cdot h(\mathbf{x}_i), \quad g_i^{(t)} = \frac{\partial \ell(y_i, F_{t-1}(\mathbf{x}_i))}{\partial F_{t-1}(\mathbf{x}_i)} \tag{21}$$

Conséquence pour l'explicabilité : Chaque arbre h_t ne modélise pas la cible y, mais le *gradient résiduel*, c'est-à-dire la direction dans laquelle le modèle précédent s'est trompé. Expliquer pourquoi $F_T(\mathbf{x}_0) = 0.87$ implique de justifier non seulement le signal principal capturé par h_1, mais également l'ensemble des micro-corrections $h_2, h_3, \ldots, h_T$, dont certaines compensent des erreurs sur des régions très locales du jeu de données. Sans régularisation stricte, le modèle mémorise du bruit : toute explication extraite refléterait alors des *artefacts statistiques locaux* plutôt que des relations causales généralisables.

4.3 Stratégies d'Explicabilité des Ensembles

On distingue deux grandes catégories de stratégies : les méthodes **post-hoc**, appliquées après l'entraînement d'un modèle opaque, et les architectures **intrinsèquement interprétables** (*glass-box*), conçues dès le départ pour être transparentes. Nous verrons également les approches hybrides issues du méta-apprentissage.

Méthodes post-hoc : limites de LIME sur les ensembles arborescents

Principe de LIME. **LIME** (Local Interpretable Model-Agnostic Explanations) tente d'expliquer la prédiction d'un modèle complexe f en ajustant localement un modèle simple g (régression Ridge ou Lasso) sur des instances perturbées autour de l'instance cible $\mathbf{x}_0$:

$$g^* = \arg\min_{g \in \mathcal{G}} \sum_{\mathbf{x}' \in Z} \pi_{\mathbf{x}_0}(\mathbf{x}') \cdot \ell\big(f(\mathbf{x}'), g(\mathbf{x}')\big) + \Omega(g) \tag{22}$$

où Z est l'ensemble des échantillons perturbés, $\pi_{\mathbf{x}_0}(\mathbf{x}')$ est un noyau de pondération gaussien qui donne plus de poids aux points proches de $\mathbf{x}_0$, et $\Omega(g)$ est une pénalité de complexité.

Algorithm 1 LIME appliqué à un classifieur d'ensemble f

Require: Instance cible $\mathbf{x}_0$, modèle f, taille d'échantillon N, noyau π
Ensure: Explication locale g^* (régression Ridge)

```
Initialiser Z ← ∅
for i = 1 à N do
    Générer x'_i par perturbation gaussienne de x_0
    Calculer ŷ_i = f(x'_i)                                  ▷ Appel au modèle boîte noire
    Calculer w_i = π_{x_0}(x'_i) = exp(−‖x_0 − x'_i‖² / σ²)
    Z ← Z ∪ {(x'_i, ŷ_i, w_i)}
end for
Résoudre : g* ← arg min_g Σ_i w_i · (f(x'_i) − g(x'_i))² + λ‖g‖_1
return g* (coefficients = importances des variables)
```

Pourquoi LIME échoue sur les ensembles arborescents. Les ensembles arborescents partitionnent l'espace des caractéristiques en *hyper-rectangles orthogonaux*, créant une fonction de décision en escalier avec de nombreuses discontinuités abruptes. Lorsque LIME génère des perturbations gaussiennes autour de $\mathbf{x}_0$, certains points perturbés tombent dans le même hyper-rectangle que $\mathbf{x}_0$ (même prédiction), d'autres traversent une frontière et obtiennent une prédiction radicalement différente.

Conséquence : l'ajustement linéaire devient extrêmement instable. Deux exécutions successives de LIME sur la même instance peuvent produire des importances de variables totalement différentes. Cette **instabilité est rédhibitoire** dans les contextes réglementaires où la reproductibilité de l'explication est une exigence légale (diagnostic clinique, octroi de crédit).

Exemple (Instabilité de LIME sur une forêt aléatoire). Supposons une forêt qui classifie les demandes de crédit. Pour le client A, une première exécution de LIME indique que le *revenu mensuel* est la variable la plus importante ; une seconde exécution (avec un autre tirage aléatoire)

indique que c'est *l'historique de remboursement*. Aucune des deux n'est fiable : la banque ne peut pas s'appuyer sur ces résultats pour justifier un refus de crédit devant un régulateur.

TreeSHAP : la solution exacte fondée sur la théorie des jeux

Valeurs de Shapley et garanties d'équité. Les **valeurs de Shapley** (SHAP), issues de la théorie des jeux coopératifs, définissent la contribution équitable de chaque variable j à la prédiction $f(\mathbf{x}_0)$:

$$\phi_j(f, \mathbf{x}) = \sum_{S \subseteq \mathcal{F} \setminus \{j\}} \frac{|S|!\,(|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} \left[f_{S \cup \{j\}}(\mathbf{x}_{S \cup \{j\}}) - f_S(\mathbf{x}_S) \right] \tag{23}$$

où $\mathcal{F}$ est l'ensemble de toutes les variables, et f_S est le modèle restreint aux variables de l'ensemble S (les variables absentes étant marginalisées).

Ces valeurs satisfont quatre axiomes fondamentaux :

1. **Efficacité :** $\sum_{j=1}^{p} \phi_j = f(\mathbf{x}) - \mathbb{E}[f(\mathbf{x})]$ (les contributions somment à la déviation par rapport à la moyenne).
2. **Symétrie :** si deux variables i et j contribuent identiquement dans toutes les coalitions, $\phi_i = \phi_j$.
3. **Nullité :** si une variable n'affecte jamais la prédiction, $\phi_j = 0$.
4. **Additivité :** pour un modèle additif $f = f_1 + f_2$, $\phi_j(f) = \phi_j(f_1) + \phi_j(f_2)$.

Problème calculatoire : L'équation (23) somme sur $2^{|\mathcal{F}|}$ sous-ensembles. Avec $p = 30$ variables, cela représente plus d'un milliard de calculs. L'approche agnostique **KernelSHAP** approxime ce calcul par échantillonnage, avec une complexité en $O(2^M)$, ce qui est rédhibitoire pour des ensembles massifs.

L'algorithme TreeSHAP : complexité polynomiale exacte. **TreeSHAP** exploite la structure arborescente pour calculer les valeurs de Shapley *exactes* avec une complexité réduite à :

$$O(T \cdot L \cdot D^2) \tag{24}$$

où T est le nombre d'arbres, L le nombre maximal de feuilles par arbre, et D la profondeur maximale.

Idée clé : Plutôt que de traiter le modèle comme une boîte noire et d'appeler $f(\cdot)$ pour chaque coalition, TreeSHAP navigue dans la structure des nœuds de chaque arbre et maintient, via de la *programmation dynamique*, le compte du poids de chaque feuille pour chaque coalition possible.

Algorithm 2 TreeSHAP — calcul exact pour un arbre unique t

Require: Arbre t, instance $\mathbf{x}_0$

Ensure: Valeurs de Shapley ϕ_j pour chaque variable j

```
Initialiser φ_j ← 0 pour tout j
PathExplainer(nœud racine, x_0, fraction = 1, zero_fraction = 1, one_fraction = 1)
procedure PathExplainer(nœud v, x_0, pf, zf, of)
    if v est une feuille then
        Mettre à jour φ_j via les poids accumulés le long du chemin
    else
        Soit j la variable de division en v et s le seuil
        h ← fraction des données passant à gauche sous v
        Appel récursif pour le sous-arbre gauche avec fractions mises à jour
        Appel récursif pour le sous-arbre droit avec fractions mises à jour
    end if
end procedure
return φ = (φ_1, ..., φ_p)
```

Table 4.1 – Comparaison des méthodes SHAP pour les ensembles arborescents

Critère	KernelSHAP	TreeSHAP	Fast TreeSHAP v2
Agnosticisme	Total (boîte noire)	Spécifique (arbres)	Spécifique (arbres)
Complexité	$O(2^M)$ approx.	$O(T \cdot L \cdot D^2)$ exact	Pré-calcul optimisé
Colinéarité	Restreinte	Paramétrable	Paramétrable
Vitesse	Très lente	Rapide	Très rapide
Usage temps réel	Non	Oui	Oui

SIRUS : extraction de règles stables depuis les forêts aléatoires

Le problème de l'instabilité des règles classiques. Une approche intuitive pour expliquer une forêt aléatoire consiste à en extraire des règles logiques de la forme SI ($x_1 > 3.5$) ET ($x_2 \leq 0.8$) ALORS $y = 1$. Cependant, les extracteurs de règles traditionnels souffrent d'une **instabilité pathologique** : un léger changement dans le jeu d'entraînement (un sous-échantillonnage bootstrap différent) modifie la racine des arbres, déclenchant une cascade de règles entièrement différentes.

SIRUS (Stable and Interpretable RUle Set) a été spécifiquement conçu pour garantir la *stabilité mathématique* des règles extraites d'une Random Forest.

Algorithm 3 Algorithme SIRUS

Require: Forêt $\mathcal{F}$ (10 000+ arbres, profondeur max 2), seuil p_0, quantiles q
Ensure: Ensemble de règles $\mathcal{R}^*$ stable et interprétable

```
Phase 1 — Discrétisation :
pour chaque variable continue x_j faire
    Calculer les q quantiles empiriques {c_{j,1}, ..., c_{j,q}}
    Remplacer x_j par sa version discrétisée x̃_j ∈ {1, ..., q}
end for                    ▷ Vocabulaire de coupures partagé entre tous les arbres
Phase 2 — Extraction des chemins :
pour chaque arbre t_k ∈ F faire
    pour chaque chemin racine-feuille π dans t_k faire
        Extraire la règle propositionnelle r_π associée à π
        Ajouter r_π au pool R
    end for
end for
Phase 3 — Filtrage par fréquence :
pour chaque règle r ∈ R faire
    p̂(r) ← fréquence d'apparition de r dans F
    if p̂(r) ≥ p_0 then
        Conserver r dans R_freq
    end if
end for
Phase 4 — Élagage et pondération :
Supprimer les règles redondantes (combinaisons linéaires de règles plus courtes)
Ajuster les poids finaux par régression Ridge : w* = arg min_w ||y − Rw||² + λ||w||²
R* ← liste finale de ∼10 règles pondérées
return R*
```

Architecture de SIRUS en quatre phases.

Garantie de stabilité via l'indice de Dice-Sørensen. La stabilité de SIRUS est mesurée par l'indice de Dice-Sørensen entre deux ensembles de règles $\mathcal{R}^{(1)}$ et $\mathcal{R}^{(2)}$ générés à partir de deux tirages indépendants du même jeu de données :

$$\mathrm{DSC}(\mathcal{R}^{(1)}, \mathcal{R}^{(2)}) = \frac{2\,|\mathcal{R}^{(1)} \cap \mathcal{R}^{(2)}|}{|\mathcal{R}^{(1)}| + |\mathcal{R}^{(2)}|} \tag{25}$$

Les études empiriques montrent que SIRUS atteint un DSC supérieur à 0.95 (contre moins de 0.5 pour les extracteurs classiques), tout en préservant une précision prédictive asymptotiquement proche de la forêt aléatoire d'origine.

Exemple (SIRUS pour le diagnostic médical). Appliqué à un jeu de données de diagnostic cardiaque (variables : âge, pression artérielle, taux de cholestérol, fréquence cardiaque maximale), SIRUS extrait typiquement un ensemble de 8 à 12 règles du type :

— *SI âge > 55 ET cholestérol > 240 ALORS risque élevé (poids : 0.31)*

— *SI fréquence_ max ≤ 120 ET pression_ art > 140 ALORS risque élevé (poids : 0.24)*

Un cardiologue peut valider chacune de ces règles médicalement, chose impossible avec les 500 arbres de la Random Forest sous-jacente.

Les architectures Glass-Box : transparence par conception

Les Explainable Boosting Machines (EBM). L'**Explainable Boosting Machine** (EBM) constitue l'avancée paradigmatique majeure : un modèle aussi performant qu'un Gradient Boosting, mais intrinsèquement interprétable. L'EBM s'inscrit dans le cadre des Modèles Additifs Généralisés avec Interactions (GA2M) dont la fonction d'hypothèse est :

$$g(\mathbb{E}[y]) = \beta_0 + \sum_{j=1}^{d} f_j(x_j) + \sum_{i \neq j} f_{ij}(x_i, x_j) \tag{26}$$

où g est la fonction de lien (logit pour la classification), $f_j(x_j)$ capture l'effet univarié de la variable j, et $f_{ij}(x_i, x_j)$ modélise les interactions paires les plus significatives.

La clé de l'interprétabilité : le Boosting cyclique (round-robin). Contrairement au Gradient Boosting classique qui entraîne des arbres multivariés profonds, l'EBM entraîne séquentiellement un micro-arbre de profondeur 1 (stump) pour une seule variable à la fois :

Algorithm 4 Entraînement d'une Explainable Boosting Machine

Require: Données $(X, \mathbf{y})$, taux η très faible, nombre d'itérations T
Ensure: Fonctions de forme $\{f_j\}_{j=1}^{d}$

```
Initialiser F(x) ← ȳ (prédiction constante)
Initialiser f_j ← 0 pour tout j
for t = 1 à T do
    Calculer les résidus : r_i ← y_i − F(x_i) pour tout i
    pour chaque variable j ∈ {1, ..., d} faire
        Entraîner un stump h_j^(t) sur (x_j, r) uniquement        ▷ Un seul axe à la fois
        Mettre à jour : f_j ← f_j + η · h_j^(t)
        Mettre à jour les résidus : r_i ← r_i − η · h_j^(t)(x_ij)
    end for
end for
return {f_j}_{j=1}^d
```

Pourquoi cela garantit l'interprétabilité : Après convergence, chaque fonction $f_j(x_j)$ est une fonction univariée exacte, visualisable sous forme d'un graphique 2D. La contribution de la variable j à la prédiction d'une instance $\mathbf{x}_0$ est exactement $f_j(x_{0j})$, sans aucune approximation.

TABLE 4.2 – Comparaison entre Gradient Boosting (XGBoost) et EBM

Métrique	Gradient Boosting (XGBoost)	Explainable Boosting Machine
Topologie	Arbres profonds multivariés	Stumps univariés (round-robin)
Interactions	Illimitées et enchevêtrées	Limitées aux paires significatives
Explicabilité	Post-hoc (TreeSHAP requis)	Intrinsèque (fonctions 2D/3D)
Fidélité de l'explication	Approximative	Exacte (100% fidélité)
Performance (AUC)	Référence	Statistiquement indiscernable

Distillation de connaissances (Knowledge Distillation). Lorsqu'un modèle opaque préexistant ne peut être remplacé par un EBM, la **distillation de connaissances** (Hinton et al., 2015) permet de transférer le savoir du modèle opaque vers un modèle transparent.

L'idée clé consiste à substituer les étiquettes binaires $y \in \{0, 1\}$ par les **probabilités molles** (soft labels) produites par le modèle enseignant (teacher) : au lieu d'apprendre $y = 1$, le modèle étudiant (student) apprend $[p_0 = 0.15, p_1 = 0.85]$. Ces probabilités molles encapsulent les *connaissances sombres* (dark knowledge) du modèle : sa confiance, la similarité qu'il perçoit entre les classes, la complexité locale de la frontière de décision.

La fonction de perte de la distillation est :

$$\mathcal{L}_{\text{dist}} = (1 - \alpha)\, \mathcal{L}_{\text{CE}}(y, \hat{y}_s) + \alpha\, T^2\, \mathcal{L}_{\text{KL}}\Big(\sigma\Big(\frac{z_t}{T}\Big)\,,\, \sigma\Big(\frac{z_s}{T}\Big)\Big) \tag{27}$$

où z_t et z_s sont les logits du teacher et du student, T est la *température* (hyperparamètre contrôlant la douceur des distributions), σ est la fonction softmax, α est un coefficient de pondération, et $\mathcal{L}_{\text{KL}}$ est la divergence de Kullback-Leibler.

Exemple (Distillation d'une forêt en un arbre décisionnel). Un hôpital dispose d'une Random Forest à 300 arbres pour prédire les réadmissions hospitalières (AUC = 0.89). Elle ne peut être remplacée car elle est intégrée dans un système critique. En appliquant la distillation :

1. La forêt génère des probabilités molles $[0.78, 0.22]$, $[0.34, 0.66]$, etc. pour chaque patient du jeu d'entraînement.
2. Un arbre de décision de profondeur 5 est entraîné sur ces probabilités molles.
3. L'arbre résultant atteint AUC = 0.86 (vs 0.89 de la forêt) mais est entièrement lisible par un médecin : 5 niveaux de règles simples.

Sélection Dynamique de Classifieurs (DCS/DES) et explicabilité contextuelle

Principe de la sélection dynamique. Les approches **DCS** (Dynamic Classifier Selection) et **DES** (Dynamic Ensemble Selection) adoptent une philosophie radicalement différente : plutôt que de combiner tous les classifieurs pour chaque instance, elles identifient dynamiquement, pour chaque nouvelle instance $\mathbf{x}_0$, les classifieurs localement les plus compétents.

L'algorithme définit une **Région de Compétence** (RoC) en projetant $\mathbf{x}_0$ dans l'espace de validation DSEL et en identifiant ses k plus proches voisins :

$$\mathrm{RoC}(\mathbf{x}_0) = \left\{\mathbf{x}_{(1)}, \ldots, \mathbf{x}_{(k)}\right\} \subset \mathcal{D}_{\mathrm{val}} \tag{28}$$

La compétence d'un classifieur h_m sur cette région est évaluée par son accuracy locale :

$$\mathrm{Comp}(h_m, \mathbf{x}_0) = \frac{1}{k} \sum_{i=1}^{k} \mathbf{1}\left[h_m(\mathbf{x}_{(i)}) = y_{(i)}\right] \tag{29}$$

Algorithm 5 Overall Local Accuracy (OLA) — DCS avec explicabilité

Require: Instance $\mathbf{x}_0$, pool de classifieurs $\{h_m\}_{m=1}^{M}$, DSEL, k
Ensure: Prédiction $\hat{y}$ et justification contextuelle

```
Trouver RoC(x_0) = k-NN(x_0, D_val)
pour chaque classifieur h_m faire
    Calculer Comp(h_m, x_0) via l'équation (29)
end for
Sélectionner h* = arg max_m Comp(h_m, x_0)
ŷ ← h*(x_0)
return ŷ, h* (expert sélectionné), RoC(x_0) (justification)
```

Avantages pour l'explicabilité. La sélection dynamique offre une propriété explicative unique : si le pool de classifieurs est composé de modèles interprétables (arbres peu profonds, régressions logistiques), le système global hérite de cette interprétabilité *par héritage*. Pour justifier la décision sur $\mathbf{x}_0$, il suffit d'expliquer :

1. Quel expert h^* a été sélectionné ;
2. Pourquoi il a été sélectionné (ses k voisins historiques dans RoC) ;
3. Quel est le raisonnement de h^* (si h^* est interprétable).

C'est ce qu'on appelle le **raisonnement par analogie** (Case-Based Reasoning) : la décision est justifiée par des exemples historiques similaires, ce qui est naturellement compréhensible par un expert métier.

XStacking : le méta-apprentissage guidé par l'explication

L'opacité fondamentale du Stacking classique. Dans un Stacking conventionnel, les prédictions des modèles de base constituent les seules entrées du méta-apprenant. Si l'on dispose de K modèles de base, le méta-apprenant opère dans un espace de dimension K : $\mathbf{z} = (\hat{y}_1, \ldots, \hat{y}_K)$. L'**espace des variables d'origine est totalement écrasé** : une méthode XAI appliquée au

méta-apprenant peut seulement dire *« le modèle A a contribué à 40% »*, mais pas *pourquoi*, ni via quelle variable d'entrée.

Architecture XStacking. **XStacking** résout ce problème en enrichissant l'espace du méta-apprenant avec les vecteurs d'importance des caractéristiques générés pour chaque modèle de base :

$$\mathbf{z}_{\text{XStacking}} = \left[\hat{y}_1, \ldots, \hat{y}_K,\ \phi(f_1(\mathbf{x})), \ldots, \phi(f_K(\mathbf{x}))\right] \tag{30}$$

où $\phi(f_k(\mathbf{x})) = (\phi_1^{(k)}, \ldots, \phi_p^{(k)})$ est le vecteur des valeurs SHAP du modèle k sur l'instance $\mathbf{x}$. L'espace d'apprentissage du méta-modèle passe de dimension K à dimension $K+K\times d = K(1+d)$ (ou $K \times d$ si les prédictions brutes sont omises).

TABLE 4.3 – Comparaison Stacking conventionnel vs XStacking

Propriété	Stacking Conventionnel	Framework XStacking
Entrées Niveau 1	Prédictions $(\hat{y}_1, \ldots, \hat{y}_K)$	Vecteurs SHAP + prédictions
Dimension méta-espace	K	$K(1+d)$
Lien variables d'origine	Coupé (out-of-fold)	Propagé nativement
Explicabilité résultante	Post-hoc, faible traçabilité	Intrinsèque, auditable
Bénéfice secondaire	Réduction de variance	Élimination biais par corrélations fallacieuses

4.4 Compromis Performance–Compréhension

Le mythe du compromis strict et sa déconstruction empirique

Pendant longtemps, l'apprentissage automatique a été dominé par l'axiome suivant : *un modèle plus performant est nécessairement moins interprétable.* Cet axiome est aujourd'hui empiriquement réfuté par plusieurs résultats convergents.

L'EBM comme preuve de concept. Des évaluations extensives sur des jeux de données critiques (prédiction de faillite bancaire, diagnostic de l'infarctus du myocarde, prédiction de risques géologiques) montrent que les EBM atteignent un AUC statistiquement indiscernable de celui de XGBoost :

TABLE 4.4 – Comparaison AUC — EBM vs XGBoost sur des jeux de données réels

Jeu de données	XGBoost	EBM	Différence
Défaut de crédit (Taiwan)	0.779	0.778	−0.001
Diagnostic cardiaque (UCI)	0.921	0.918	−0.003
Mortalité soins intensifs	0.862	0.859	−0.003
Détection fraude bancaire	0.974	0.971	−0.003

Ces différences sont statistiquement non significatives et bien inférieures aux variations dues aux hyperparamètres ou à la variabilité d'échantillonnage. Le compromis n'est donc pas une loi absolue, mais un artefact des architectures d'apprentissage antérieures.

L'effet Rashomon et la multiplicité des modèles équivalents

Définition formelle de l'ensemble de Rashomon. Le phénomène de **multiplicité des modèles** (*Model Multiplicity* ou *Rashomon Effect*), formalisé par Leo Breiman, constitue l'une des découvertes les plus déstabilisantes de l'apprentissage moderne : dans des espaces à haute dimension, il existe souvent un très grand nombre de modèles statistiquement équivalents, mais reposant sur des logiques décisionnelles totalement incompatibles.

Définition (Ensemble de Rashomon $\mathcal{R}(\epsilon)$). Soit f^* le modèle optimal dans l'espace des hypothèses $\mathcal{F}$, de perte empirique $\mathcal{L}(f^*)$. L'**ensemble de Rashomon** est défini par :

$$\mathcal{R}(\epsilon) = \left\{ f \in \mathcal{F} \mid \mathcal{L}(f) \leq \mathcal{L}(f^*) + \epsilon \right\} \tag{31}$$

Tous les modèles de $\mathcal{R}(\epsilon)$ ont un pouvoir de généralisation pratiquement identique, mais peuvent s'appuyer sur des ensembles de variables et des logiques de décision radicalement différents.

Implication pour l'explicabilité : l'incertitude interprétative. Appliquer SHAP ou un PDP (Partial Dependence Plot) sur le *meilleur modèle unique* génère un **biais cognitif dangereux** : cela donne l'illusion que le profil causal obtenu est *la seule vérité* sous-jacente aux données. Or, des centaines d'autres modèles de performances équivalentes pourraient produire des explications contradictoires.

La solution : le Rashomon PDP. On agrège les Graphiques de Dépendance Partielle (PDP) sur l'ensemble $\mathcal{R}(\epsilon)$. Cela produit :

— **Stabilité concordante :** si l'effet d'une variable x_j est positif sur la quasi-totalité des modèles de $\mathcal{R}(\epsilon)$, l'explication est certifiée robuste et indépendante du choix d'optimisation.

— **Désaccord interne :** si les PDP sont très dispersés entre les modèles de $\mathcal{R}(\epsilon)$, le système signale explicitement qu'il n'existe *pas de consensus* dans les données sur le rôle de cette variable. Toute conclusion causale serait une sur-interprétation.

Exemple (Rashomon PDP en oncologie). Un modèle de prédiction de survie au cancer du sein est entraîné sur des variables biologiques (expression génique, âge, stade). Le Rashomon PDP révèle que pour la variable *âge* :

— 95% des modèles de $\mathcal{R}(0.01)$ concordent : l'âge augmente le risque au-delà de 60 ans. → Explication certifiée robuste, actionnable cliniquement.

— Pour la variable *marqueur HER2* : 50% des modèles indiquent un effet protecteur, 50% un effet délétère. → Aucune conclusion causale ne doit être tirée ; une étude clinique dédiée est nécessaire.

Le Consensus XAI : agréger les explications pour réduire leur variance

La méthode NormEnsembleXAI. Tout comme l'apprentissage d'ensemble stabilise les prédictions, le **Consensus XAI** stabilise les explications en agrégeant les vecteurs d'importance produits par différentes méthodes ou différentes exécutions. Le framework **NormEnsembleXAI** structure ce processus en trois étapes :

Algorithm 6 NormEnsembleXAI — agrégation d'explications plurielles

Require: Matrices d'explications $\{\Phi^{(1)}, \ldots, \Phi^{(K)}\}$ de sources distinctes
Ensure: Explication consensuelle stable $\bar{\Phi}$

1: **Étape 1 — Normalisation par méthode :**
2: **pour chaque** matrice $\Phi^{(k)}$ **faire**
3: Appliquer une standardisation robuste :
4: $\tilde{\phi}_j^{(k)} \leftarrow \dfrac{\phi_j^{(k)} - \text{median}(\Phi^{(k)})}{\text{P}_{99}(\Phi^{(k)}) - \text{P}_{01}(\Phi^{(k)})}$ ▷ Insensible aux valeurs aberrantes
5: **end for**
6: **Étape 2 — Harmonisation ordinale :**
7: Vérifier que la normalisation préserve les ordres de rang entre variables
8: **Étape 3 — Agrégation :**
9: Calculer le score de fiabilité : $s_k = \text{corr}(\Phi^{(k)}, f(\mathbf{x}))$
10: $\bar{\Phi} \leftarrow \sum_{k=1}^{K} s_k \cdot \tilde{\Phi}^{(k)} \Big/ \sum_{k=1}^{K} s_k$
11: **return** $\bar{\Phi}$

Explanation Consistency Training (ECT). L'**Explanation Consistency Training** (ECT) va encore plus loin : au lieu d'agréger les explications après l'entraînement, ECT intègre la *cohérence explicative* directement dans la fonction de perte lors de l'entraînement. La perte totale est :

$$\mathcal{L}_{\text{ECT}} = \mathcal{L}_{\text{pred}} + \lambda_{\text{cons}} \cdot \frac{1}{N} \sum_{i=1}^{N} \|\phi(f, \mathbf{x}_i) - \phi(f, \mathbf{x}_i + \epsilon_i)\|^2 \tag{32}$$

Le second terme pénalise les modèles dont les explications changent trop fortement sous de petites perturbations ϵ_i des données d'entrée. L'ECT force l'ensemble à converger non seulement vers une *exactitude unanime* (même prédiction), mais surtout vers une *cohérence causale* (même justification). Des études sur des données médicales (MIMIC-IV) montrent des améliorations de cohérence supérieures à 120% sans dégradation significative de la performance prédictive.

Guide de décision : quelle stratégie adopter ?

TABLE 4.5 – Guide de sélection de la stratégie d'explicabilité selon le contexte

Contexte	**Contrainte principale**	**Méthode recommandée**	**Justification**
Nouveau projet, domaine critique	Performance + légalité	EBM (GA2M)	Interprétabilité native, AUC comparable
Modèle existant, irremplaçable	Compatibilité	TreeSHAP + Distillation	Analyse locale précise
Audit réglementaire	Stabilité	SIRUS	Règles stables (DSC > 0.95)
Décision sur instances rares	Contexte local	DCS/DES	Expertise locale certifiée
Haute dimensionnalité, $p > 100$	Cohérence globale	Consensus XAI + Rashomon	Certifie la robustesse causale
Stacking existant	Traçabilité	XStacking	Lien bout-en-bout restauré

4.5 Conclusion

Ce chapitre a mis en lumière que l'opacité des ensembles de classifieurs n'est pas un simple défaut technique, mais la conséquence directe des mécanismes mathématiques qui fondent leur supériorité : explosion de la dimension VC, exploitation délibérée de la diversité (identité de Krogh-Vedelsby), accumulation de corrections résiduelles (Boosting).

Face à cette complexité, l'ingénierie de l'explicabilité a développé un spectre de solutions complémentaires :

1. **Post-hoc analytique :** TreeSHAP calcule des valeurs de Shapley exactes en temps polynomial, en exploitant la structure des arbres.
2. **Distillation symbolique :** SIRUS extrait des règles SI-ALORS stables depuis les forêts aléatoires, sans perte significative de précision.
3. **Architectures glass-box :** Les EBM montrent empiriquement qu'interprétabilité et performance peuvent coexister sans compromis significatif.
4. **Explicabilité contextuelle :** La sélection dynamique (DCS/DES) offre une justification par analogie, adaptée aux instances localement complexes.

5. **Méta-apprentissage transparent :** XStacking restaure la traçabilité bout-en-bout du Stacking en enrichissant le méta-espace avec les vecteurs SHAP.
6. **Robustesse de l'explication :** Le Consensus XAI et les ensembles de Rashomon quantifient l'incertitude interprétative et certifient la stabilité causale.

La frontière entre performance et compréhension, longtemps perçue comme infranchissable, se révèle être le résultat d'une ingénierie insuffisante plutôt qu'une limite théorique absolue. L'avenir des systèmes critiques repose sur des architectures où explicabilité et précision se renforcent mutuellement, comme l'illustrent XStacking et les EBM.

5 Conclusion générale

Ce rapport a exploré deux piliers de l'apprentissage automatique contemporain : la **combinaison de classifieurs** et l'**explicabilité des modèles**, ainsi que leurs interactions. Plusieurs enseignements en découlent.

La combinaison de classifieurs (bagging, boosting, stacking, voting) permet d'améliorer la précision et la robustesse des prédictions en s'appuyant sur la diversité et l'agrégation des modèles. Les motivations théoriques (réduction de la variance, du biais ou des deux) et les choix architecturaux (génération de l'ensemble, sélection, fusion) conditionnent directement les performances. En contrepartie, ces systèmes deviennent plus complexes : plusieurs apprenants, mécanismes de pondération ou de méta-apprentissage, et donc une décision plus difficile à décomposer et à justifier.

L'explicabilité apparaît comme une exigence complémentaire à la performance. La distinction entre interprétabilité (structure du modèle lisible) et explicabilité (capacité à rendre compte des décisions) permet de clarifier les objectifs. Les méthodes post-hoc (importance des variables, LIME, SHAP, PDP/ICE) offrent des leviers pour expliquer des modèles a priori opaques, en local ou en global. Leur utilisation dans les systèmes décisionnels est renforcée par les enjeux éthiques et réglementaires (transparence, non-discrimination, droit à l'information).

À l'intersection des deux thèmes, le rapport a mis en évidence le **problème de la boîte noire** propre aux ensembles : une décision collective est plus difficile à attribuer à des facteurs précis qu'une décision d'un modèle unique. Les stratégies d'explicabilité des ensembles (explication au niveau de l'agrégation, au niveau des apprenants de base, ou par approximation) permettent de progresser, mais un **compromis entre performance et compréhension** reste inévitable. Selon le contexte (réglementation, domaine critique, acceptabilité utilisateur), le curseur entre complexité du modèle et niveau d'explicabilité requis devra être ajusté.

En synthèse, la combinaison de classifieurs reste un outil puissant pour la qualité prédictive ; l'explicabilité en est le corollaire indispensable dès que les décisions ont un impact humain ou juridique. Les travaux futurs pourront approfondir l'intégration de contraintes d'explicabilité dès la conception des systèmes d'ensemble et l'évaluation standardisée des méthodes d'explication dans ce cadre.

Bibliographie

[1] Analytics Vidhya. Bagging, boosting and stacking : Ensemble learning in ml models. https://www.analyticsvidhya.com/blog/2023/01/ensemble-learning-methods-bagging-boosting-and-stacking/, 2023. Consulté le 23 février 2026.

[2] Leo Breiman. Bagging predictors. *Machine Learning*, 24(2) :123–140, 1996. Article fondateur du bagging.

[3] Leo Breiman. Stacked regressions. *Technical Report*, 1996. Manuscrit disponible via les archives de Leo Breiman.

[4] Jason Brownlee. Bagging vs boosting vs stacking : Which ensemble method wins in 2025 ? https://machinelearningmastery.com/bagging-vs-boosting-vs-stacking-which-ensemble-method-wins-in-2025/, 2025. Consulté le 23 février 2026.

[5] Tianqi Chen and Carlos Guestrin. Xgboost : A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[6] Codefinity. Learn voting : Hard and soft. https://codefinity.com/courses/v2/3172b303-2b09-492b-8bb9-5bbd1bdcef09/81f3a01b-d537-4478-bc23-3ea1895e4f1f/d66aa7bd-877e-4808-ad40-aa051eabbc4a, 2024. Consulté le 23 février 2026.

[7] William W. Cohen. Ensemble methods for machine learning. https://www.cs.cmu.edu/~wcohen/10-601/ensembles.pdf, 2010. Support de cours, Carnegie Mellon University. Consulté le 23 février 2026.

[8] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti. Dynamic ensemble selection and instantaneous pruning for regression. In *ESANN 2014*, 2014. Consulté le 23 février 2026.

[9] R. M. O. Cruz, R. Sabourin, and G. D. C. Cavalcanti. Deslib : A dynamic ensemble selection library in python. *Journal of Machine Learning Research*, 21(8) :1–5, 2020. Consulté le 23 février 2026.

[10] DESlib project. Welcome to deslib documentation ! https://deslib.readthedocs.io/, 2020. Consulté le 23 février 2026.

[11] Luis Fred. Multiple classifier systems — a brief introduction. https://medium.com/luisfredgs/multiple-classifier-systems-a-brief-introduction-71238d9c794f, 2023. Consulté le 23 février 2026.

[12] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1) :119–139, 1997.

[13] Jerome H. Friedman. Greedy function approximation : A gradient boosting machine. *Annals of Statistics*, 29(5) :1189–1232, 2001.

[14] GeeksforGeeks. A comprehensive guide to ensemble learning. `https://www.geeksforgeeks.org/machine-learning/a-comprehensive-guide-to-ensemble-learning/`, 2023. Consulté le 23 février 2026.

[15] GeeksforGeeks. Voting classifier. `https://www.geeksforgeeks.org/machine-learning/voting-classifier/`, 2023. Consulté le 23 février 2026.

[16] Vibhav Gogate. Bias/variance tradeoff and ensemble methods. `https://personal.utdallas.edu/~vibhav.gogate/ml/2020f/lectures/EnsembleMethods.pdf`, 2020. Support de cours, Université du Texas à Dallas. Consulté le 23 février 2026.

[17] Great Learning. Understanding the ensemble method bagging and boosting. `https://www.mygreatlearning.com/blog/bagging-boosting/`, 2023. Consulté le 23 février 2026.

[18] Laurent Heutte. Combinaison de classifieurs. `http://lheutte.free.fr/rdf/ArchiCombi.pdf`, 2003. Consulté le 23 février 2026.

[19] IBM. What is bagging ? `https://www.ibm.com/think/topics/bagging`, 2024. Consulté le 23 février 2026.

[20] IBM. What is ensemble learning ? `https://www.ibm.com/think/topics/ensemble-learning`, 2024. Consulté le 23 février 2026.

[21] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. Lightgbm : A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems 30*, pages 3146–3154, 2017.

[22] Albert Ko, Robert Sabourin, and Alceu de Souza Britto Jr. From dynamic classifier selection to dynamic ensemble selection. In *Proceedings of the 7th International Conference on Machine Learning and Applications*, 2007. Consulté le 23 février 2026.

[23] Anders Krogh and Jesper Vedelsby. Neural network ensembles, cross validation, and active learning. In *Advances in Neural Information Processing Systems 7 (NIPS 1994)*, pages 231–238, 1995. Article fondateur de l'identité de Krogh et Vedelsby sur l'ambiguïté des ensembles.

[24] Ludmila I. Kuncheva and Juan J. Rodríguez. A unified theory of diversity in ensemble learning. *Information Fusion*, 2023. Résumé disponible sur ResearchGate, consulté le 23 février 2026.

[25] G. Metzler. Machine learning td – méthodes ensemblistes. `https://eric.univ-lyon2.fr/gmetzler/courses/contents/machinelearning/td4.pdf`, 2022. Travaux dirigés, Université Lyon 2. Consulté le 23 février 2026.

[26] NS Academy. What is ensemble learning (bagging vs boosting vs stacking) ? `https://medium.com/@sharetonschool/what-is-ensemble-learning-bagging-vs-boosting-vs-stacking-c3c8cc94a3d5`, 2023. Consulté le 23 février 2026.

[27] Robert E. Schapire. A brief introduction to boosting. *IJCAI*, 99 :1401–1406, 1999.

[28] scikit-learn-contrib. Deslib : A python library for dynamic classifier and ensemble selection. `https://github.com/scikit-learn-contrib/DESlib`, 2020. Consulté le 23 février 2026.

[29] Kai Ming Ting and Ian H. Witten. Issues in stacked generalization. *Journal of Artificial Intelligence Research*, 10 :271–289, 1999.

[30] Turing College. Ensemble learning : Key techniques explained. `https://www.turingcollege.com/blog/ensemble-learning-2`, 2024. Consulté le 23 février 2026.

[31] Ultralytics. Exploration de l'apprentissage d'ensemble : son rôle dans l'ia et la ml. `https://www.ultralytics.com/fr/blog/exploring-ensemble-learning-and-its-role-in-ai-and-ml`, 2024. Consulté le 23 février 2026.

[32] Wikipedia. Bias–variance tradeoff. `https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff`, 2026. Consulté le 23 février 2026.

[33] Wikipedia. Ensemble learning. `https://en.wikipedia.org/wiki/Ensemble_learning`, 2026. Consulté le 23 février 2026.

[34] Wikipédia. Théorème du jury. `https://fr.wikipedia.org/wiki/Th%C3%A9or%C3%A8me_du_jury`, 2026. Consulté le 23 février 2026.

[35] David H. Wolpert. Stacked generalization. *Neural Networks*, 5(2) :241–259, 1992.

[36] Kaleemullah Younas. Bagging, boosting and stacking : Ensemble learning in ml models. `https://medium.com/@kaleemullahyounas123/bagging-boosting-and-stacking-ensemble-learning-in-ml-models-8eb2f73afb07`, 2023. Consulté le 23 février 2026.